计算机组成原理与系统结构



第8章 RISC-V与ARM模型机设计实例

杭州電子科技大學
HANGZHOU DIANZI UNIVERSITY

第8章 RISC-V与ARM模型机设计实例

- 8.1 RISC-V模型机的设计
- 8.2 ARM模型机的设计
- 本章小结

8.1 RISC-V模型机的设计

- 8.1.1 RISC-V模型机的目标指令集
- 8.1.2 RISC-V模型机的部件设计
- 8.1.3 运算及传送指令的数据通路设计
- 8.1.4 访存指令的数据通路设计
- 8.1.5 转移类指令的数据通路设计
- 8.1.6 RISC-V模型机控制单元CU的设计

- 确定指令系统
- 确定系统结构
  - 功能部件
  - CPU类型：多周期
  - 连接方式：直接连接
- 建立数据通路，分析指令执行过程
- 设计控制单元

8.1.1 RISC-V模型机的目标指令集

- 选择实现RV32I的核心指令子集：25条

指令类型	指令	opcode(7)	指令格式
R型运算指令10条	add rd,rs1,rs2	0110011	R
	sub rd,rs1,rs2	0110011	R
	sll rd,rs1,rs2	0110011	R
	slt rd,rs1,rs2	0110011	R
	sltu rd,rs1,rs2	0110011	R
	xor rd,rs1,rs2	0110011	R
	srl rd,rs1,rs2	0110011	R
	sra rd,rs1,rs2	0110011	R
	or rd,rs1,rs2	0110011	R
	and rd,rs1,rs2	0110011	R

指令类型	指令	opcode(7)	指令格式
I型运算指令9条	addi rd,rs1,imm12	0010011	I
	slli rd,rs1,shamt	0010011	I
	slti rd,rs1,imm12	0010011	I
	sltiu rd,rs1,imm12	0010011	I
	xori rd,rs1,imm12	0010011	I
	srli rd,rs1,shamt	0010011	I
	srai rd,rs1,shamt	0010011	I
	ori rd,rs1,imm12	0010011	I
	andi rd,rs1,imm12	0010011	I
传送指令	lui rd,imm20	0110111	U
访存指令2条	lw rd, imm12(rs1)	0000011	I
	sw rs2, imm12(rs1)	0100011	S
转移指令3条	jal rd,offset20	1101111	J
	jalr rd,offset12(rs1)	1100111	I
	beq rs1,rs2,offset12	1100011	B

分析指令格式和各指令操作

- 指令执行过程：
  - 取指令并译码：根据PC从内存取出指令，PC+4→PC，译码
  - 执行指令：
    - 首先根据指令字的字段，读取1~2个寄存器（rs1、rs2）内容
    - 再根据指令具体功能，做不同的操作：
      - 算术逻辑类指令：ALU运算，写寄存器
      - 传送类指令： 生成立即数，写寄存器
      - 访存指令：计算存储器地址（ALU运算），读存储器或者写存储器
      - 转移类指令：比较数据（ALU运算），PC+偏移量→PC

指令执行过程举例

指令举例	格式	功能	执行过程				
add rd,rs1,rs2	R	rs1+ rs2→rd	取指令	读寄存器 rs1、rs2	ALU运算	写寄存器 rd	
addi rd,rs1,imm12	I	rs1+ SE32(imm12)→rd	取指令	读寄存器 rs1	ALU运算	写寄存器 rd	
lui rd,imm20	U	{imm20,12{0}}→rd	取指令	写寄存器rd			
lw rd, imm12(rs1)	I	Mem32[rs1+SE32(imm12)]→rd	取指令	读寄存器 rs1	ALU计算EA	读存储器	写寄存器rd
sw rs2, imm12(rs1)	S	rs2→Mem32[rs1+SE32(imm12)]	取指令	读寄存器 rs1	ALU计算EA	写存储器	
jal rd,offset20	J	PC+4→rd,PC+SE32(offset20*2)→PC	取指令	写寄存器rd，写PC			
jalr rd,offset12(rs1)	I	PC+4→rd,rs1+SE32(offset12)→PC	取指令	读寄存器 rs1	计算转移地址	写寄存器，写PC	
beq rs1,rs2,offset12	B	if(rs1=rs2)PC+SE32(offset12*2)→PC	取指令	读寄存器 rs1、rs2	ALU减法运算	有条件写PC	

指令执行过程举例——如果考虑流水线

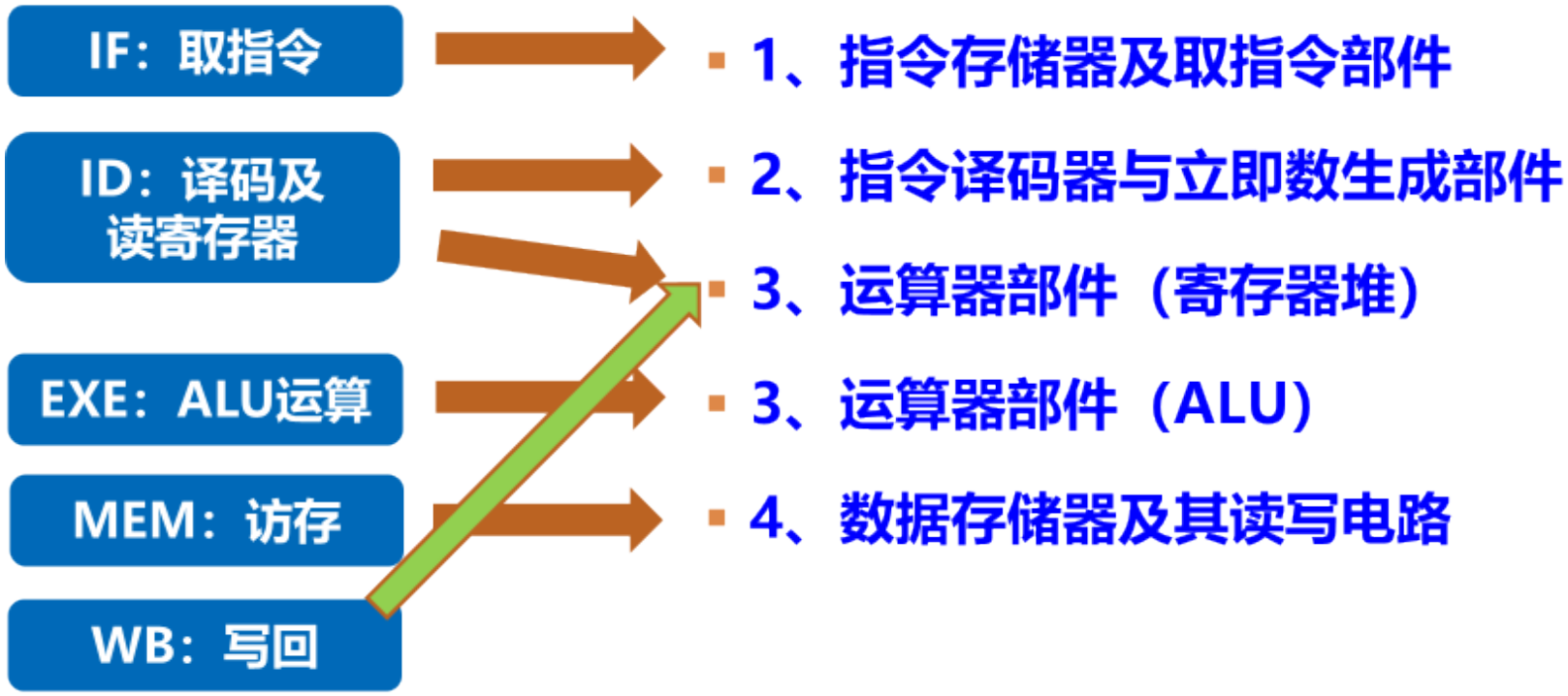
指令	格式	功能	取指令	读寄存器	ALU运算	访存	写回
add rd,rs1,rs2	R	rs1+ rs2→rd	取指令	读寄存器 rs1、rs2	ALU运算		写寄存器 rd
addi rd,rs1,imm12	I	rs1+ SE32(imm12)→rd	取指令	读寄存器 rs1	ALU运算		写寄存器 rd
lui rd,imm20	U	{imm20,12{0}}→rd	取指令				写寄存器 rd
lw rd, imm12(rs1)	I	Mem32[rs1+SE32 (imm12)]→rd	取指令	读寄存器 rs1	ALU计算 EA	读存储器	写寄存器 rd
sw rs2, imm12(rs1)	S	rs2→Mem32[rs1+ SE32(imm12)]	取指令	读寄存器 rs1	ALU计算 EA	写存储器	
jal rd,offset20	J	PC+4→rd,PC+SE3 2(offset20*2)→PC	取指令				写寄存器 rd，写PC
jalr rd,offset12(rs1)	I	PC+4→rd,rs1+SE3 2(offset12)→PC	取指令	读寄存器 rs1	计算转移地址		写寄存器，写PC
beq rs1,rs2,offset12	B	if(rs1=rs2)PC+SE3 2(offset12*2)→PC	取指令	读寄存器 rs1、rs2	ALU减法运算		有条件写 PC



IF ID EX MEM WB

8.1.2 RISC-V模型机的部件设计

- 为实现RV32I的核心指令子集，设计方案：
  - （1）多周期CPU
  - （2）直接连接（无总线）
  - （3）存储器采用哈佛结构
- 哪些部件?



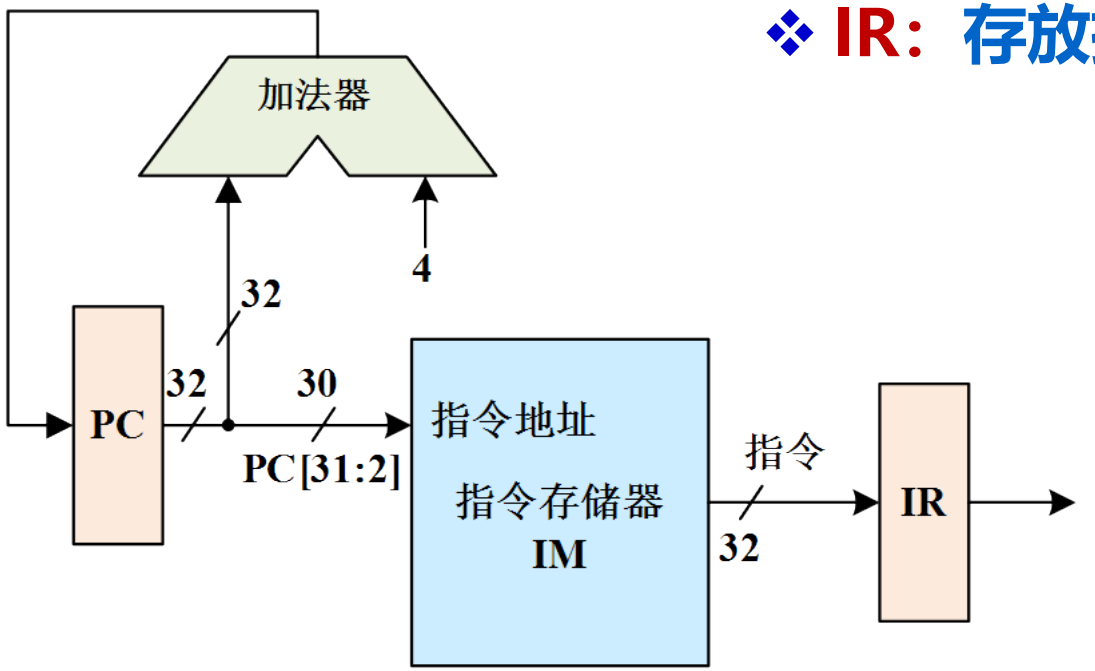
8.1.2 RISC-V模型机的部件设计

- 1、指令存储器及取指令部件
  - 指令存储器
  - PC、IR
- 2、指令译码器与立即数生成部件
  - 指令初级译码器ID1
  - 指令二级译码器ID2
- 3、运算器部件
  - 通用寄存器堆Regs
  - ALU、FR、暂存器
- 4、数据存储器

1、指令存储器及取指令部件——取指令IF

- 指令执行第一阶段：取指令
  - Mem[PC]→IR
  - PC自增

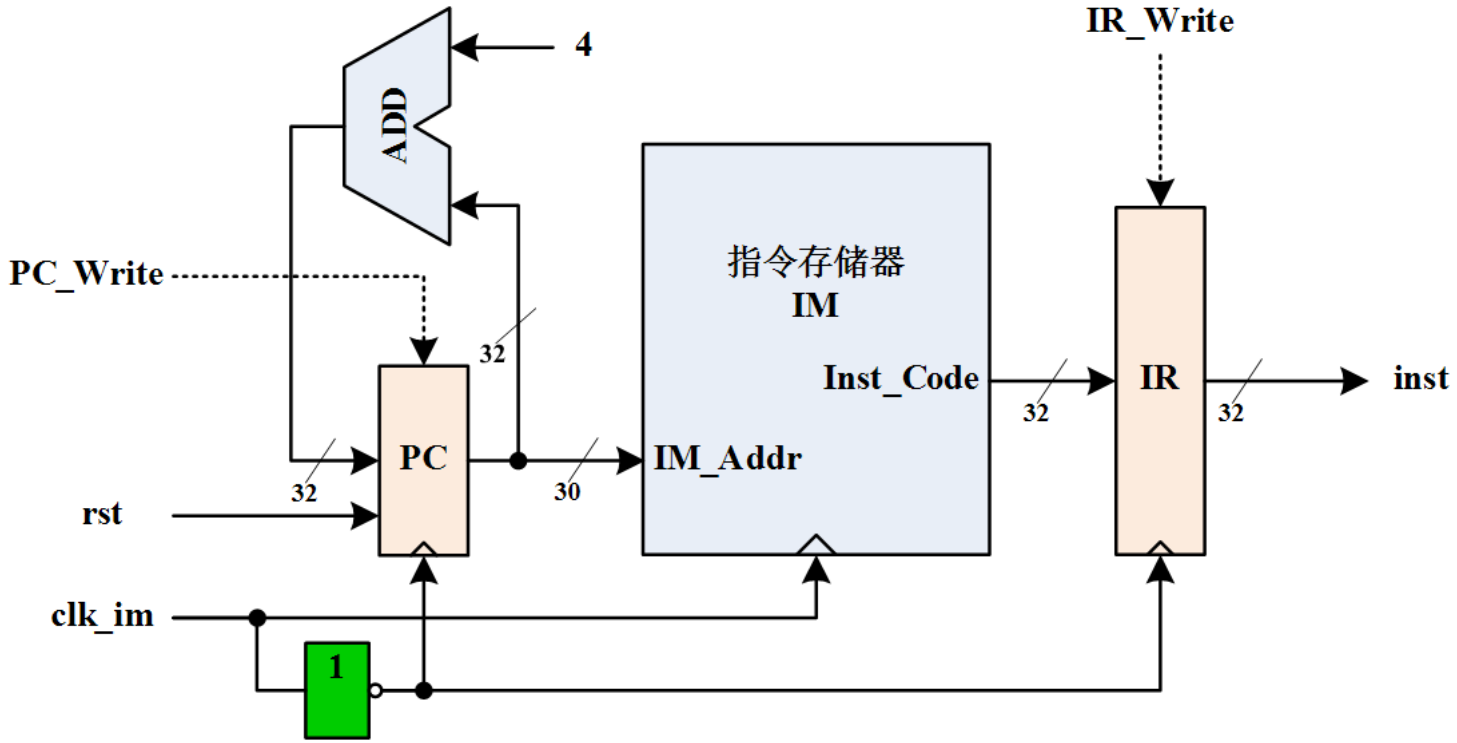
- 存储器结构：哈佛结构 or 普林斯顿结构?
- PC自增量是多少？怎么实现?
- 指令存储器：只读存储器；按字节编址；小端模式
- PC：存放指令地址，32位，取指令后自动增4
  - 自增4：如何实现? → 独立的加法器
- IR：存放指令代码，32位



1、指令存储器及取指令部件——取指令IF

- 取指令：
  - Mem[PC]→IR
  - PC自增
- 如何保证在取指令时，是按照PC当前值作为地址，而不是PC+4作为地址?
- 即：时序控制?

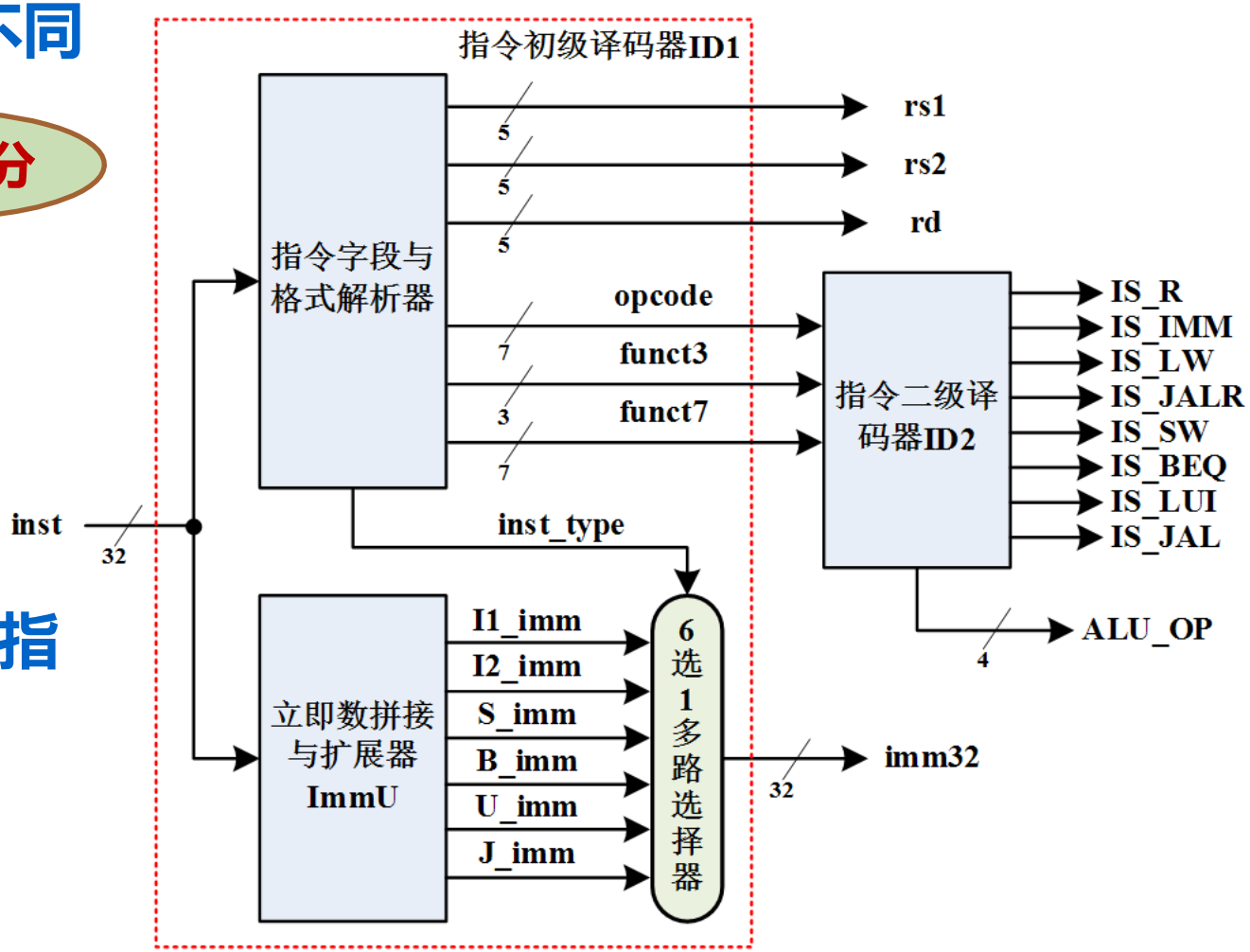
- clk上跳沿：取指令，即读存储器
- clk下跳沿：
  - 指令置入IR：IM读出的指令码打入IR
  - 更新PC的值：PC+4 打入PC



2、指令译码器与立即数生成部件——指令译码ID

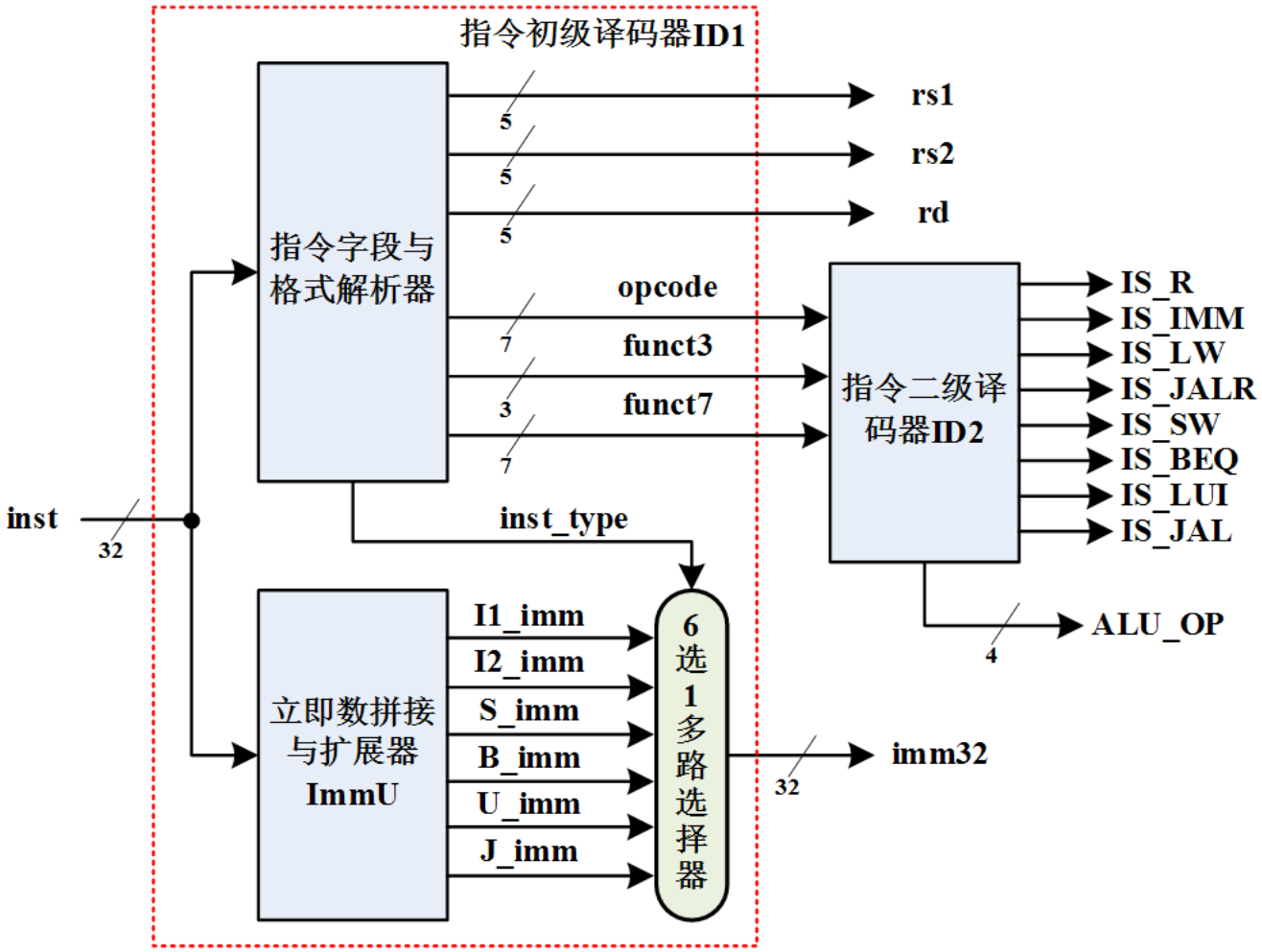
- 指令执行第二阶段：分析指令
  - 由指令译码器对指令码进行解析，分辨出是哪一条指令，并识别出操作数类型及寻址方式
- RISC-V：6种指令格式，字段排列不同
- 指令译码分为两个阶段
  - （1）初级译码器ID1：
    - ①指令字段与格式解析器
    - ②立即数生成器
  - （2）指令二级译码器ID2：产生指令信号、ALU_OP

简单拆分



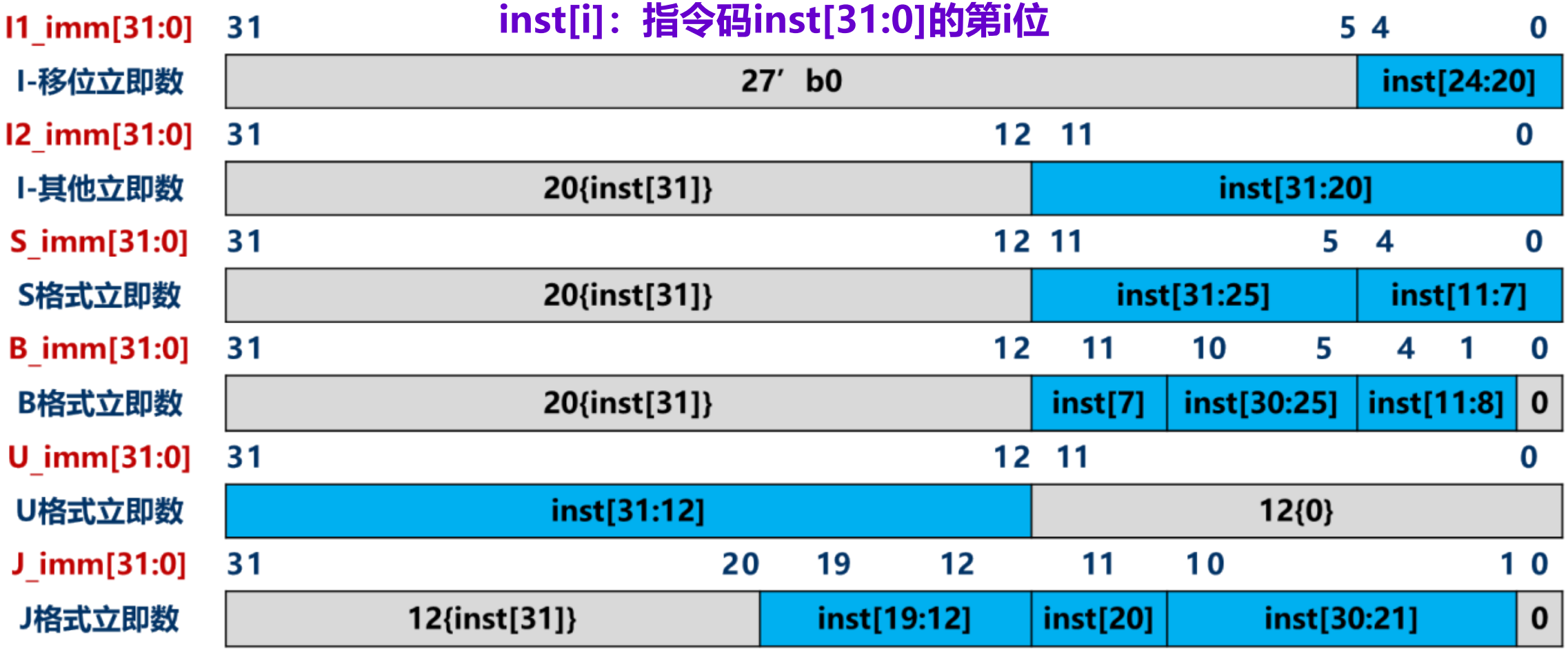
②立即数生成器

- 1）立即数拼接与生成器：
  - 按照各格式，对立即数进行拼接与扩展，产生6个32位立即数
- 2）6选1多路选择器：
  - 按照当前指令的格式，在6个立即数中选择一个输出，记为imm32



立即数拼接与扩展器

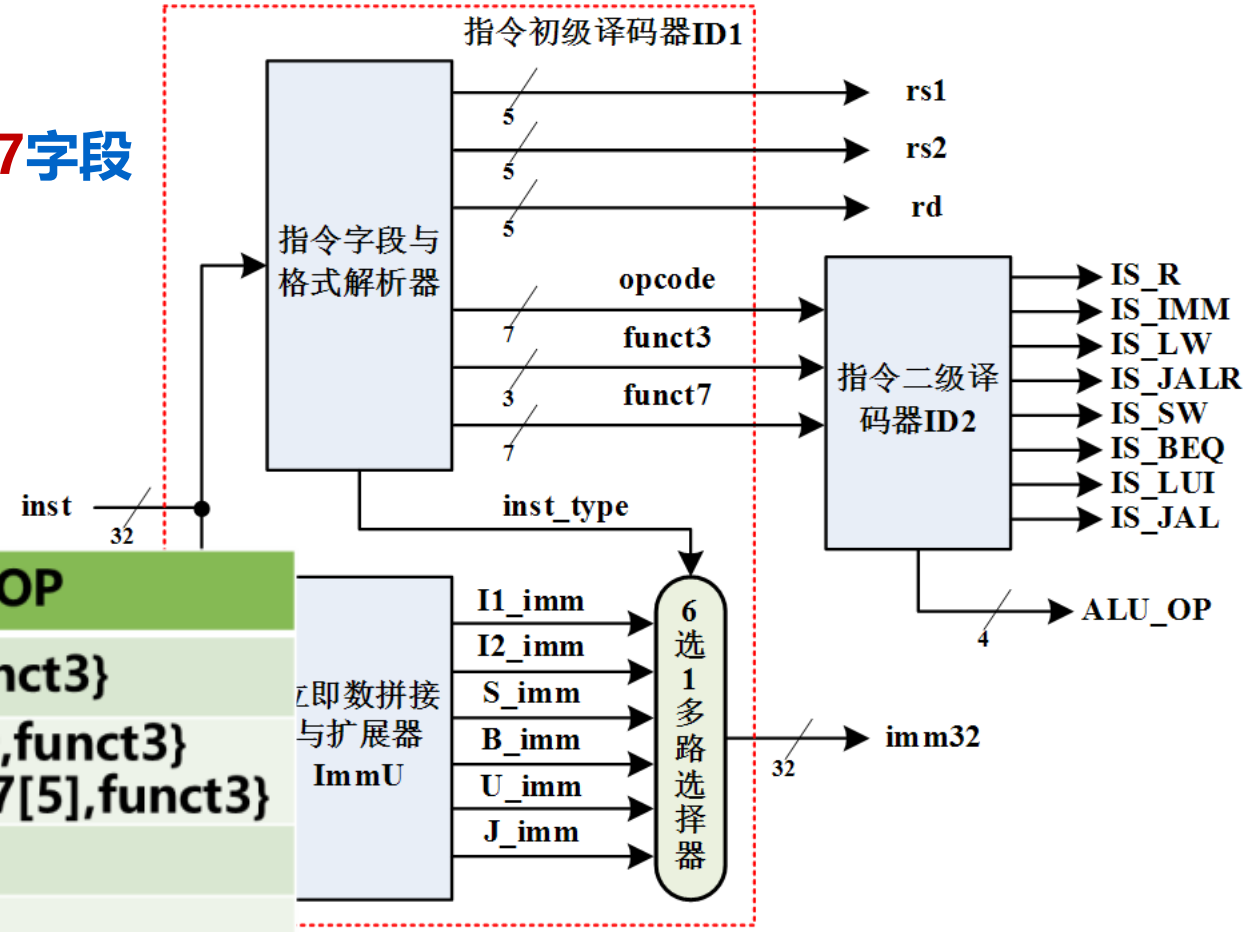
inst[i]：指令码inst[31:0]的第i位




```
assign B_imm={20{inst[31]},inst[7],inst[30:25],inst[11:8],1’ b0}
```

（2）指令二级译码器ID2

- ID2：组合逻辑电路
  - 输入：ID1产生的opcode、funct3和funct7字段
  - 输出：
    - 指令信息：8个（对应不同的opcode）
    - 运算码：ALU_OP，4位



输入：opcode	输出：指令信息	输出：ALU_OP
0110011	IS_R	{funct7[5],funct3}
0010011	IS_IMM	funct3≠101：{1’ b0,funct3} funct3=101：{funct7[5],funct3}
0110111	IS_LUI	xxxx
0000011	IS_LW	0000
0100011	IS_SW	0000
1101111	IS_JAL	0000
1100111	IS_JALR	0000
1100011	IS_BEQ	1000

（2）指令二级译码器ID2

❖ALU_OP怎么产生?

❖R型指令为例：

指令	funct7	funct3	指令运算操作	ALU_OP
add rd,rs1,rs2	0000000	000	算术加	0000
sub rd,rs1,rs2	0100000	000	算术减	1000
sll rd,rs1,rs2	0000000	001	左移	0001
slt rd,rs1,rs2	0000000	010	有符号数小于置位	0010
sltu rd,rs1,rs2	0000000	011	无符号数小于置位	0011
xor rd,rs1,rs2	0000000	100	位异或	0100
srl rd,rs1,rs2	0000000	101	逻辑右移	0101
sra rd,rs1,rs2	0100000	101	算术右移	1101
or rd,rs1,rs2	0000000	110	位或	0110
and rd,rs1,rs2	0000000	111	位与	0111

R型： assign ALU_OP = {funct7[5],funct3};

（2）指令二级译码器ID2

❖ALU_OP怎么产生?　　❖I型运算指令：和R型指令相同吗?

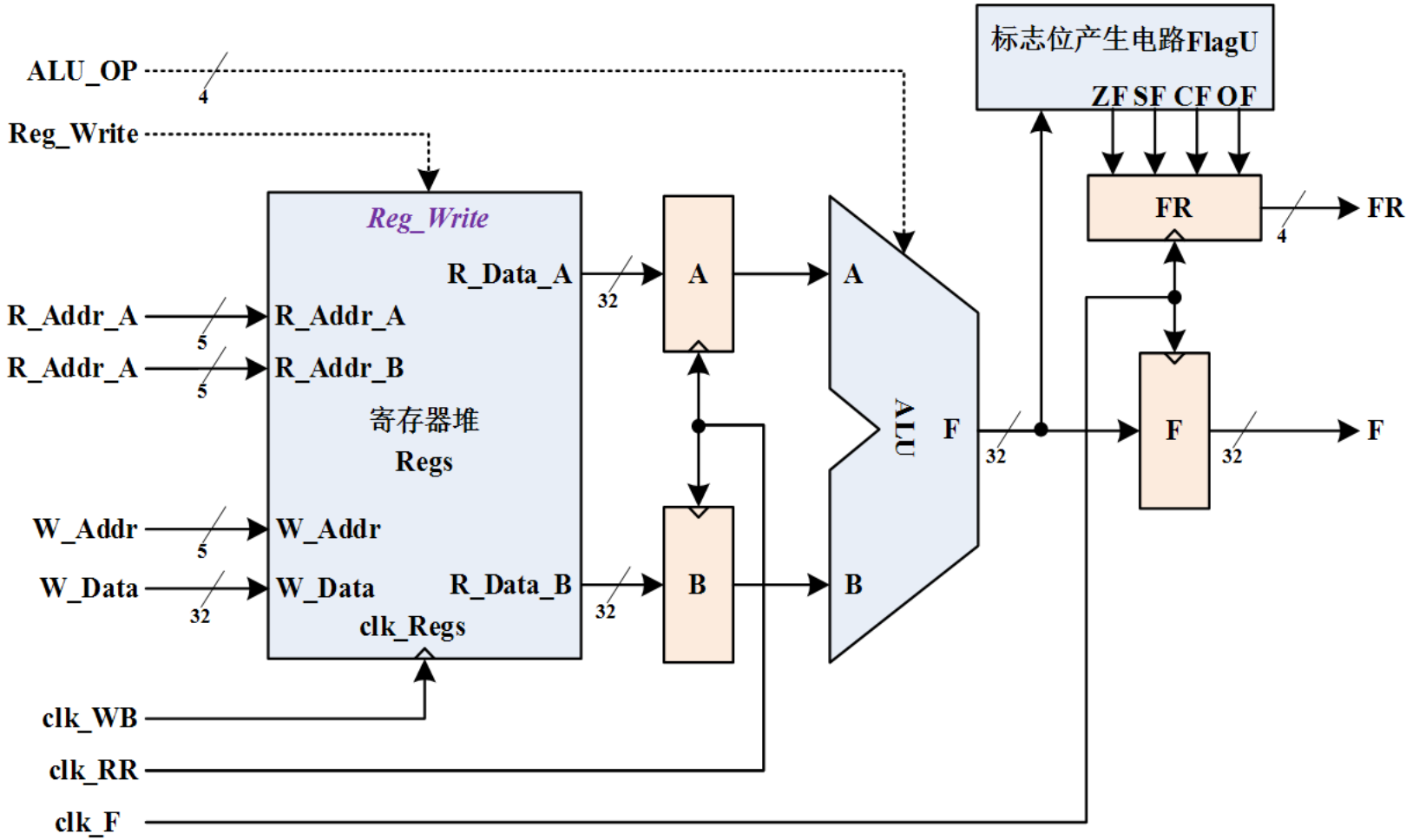
指令	funct7	funct3	指令运算操作	ALU_OP
addi rd,rs1,imm12	—	000	算术加	0000
slli rd,rs1,shamt	0000000	001	左移	0001
slti rd,rs1,imm12	—	010	有符号数小于置位	0010
sltiu rd,rs1,imm12	—	011	无符号数小于置位	0011
xori rd,rs1,imm12	—	100	位异或	0100
srli rd,rs1,shamt	0000000	101	逻辑右移	0101
srai rd,rs1,shamt	0100000	101	算术右移	1101
ori rd,rs1,imm12	—	110	位或	0110
andi rd,rs1,imm12	—	111	位与	0111

```
I型： assign ALU_OP = (funct3==3'b101) ? {funct7[5],funct3}:{1'b0,funct3};
```

3、运算器部件——执行EX

- 指令执行的第三个阶段：执行操作
  - 依据：指令的功能
  - 操作：运算、传送、存储器或IO设备的读写操作等

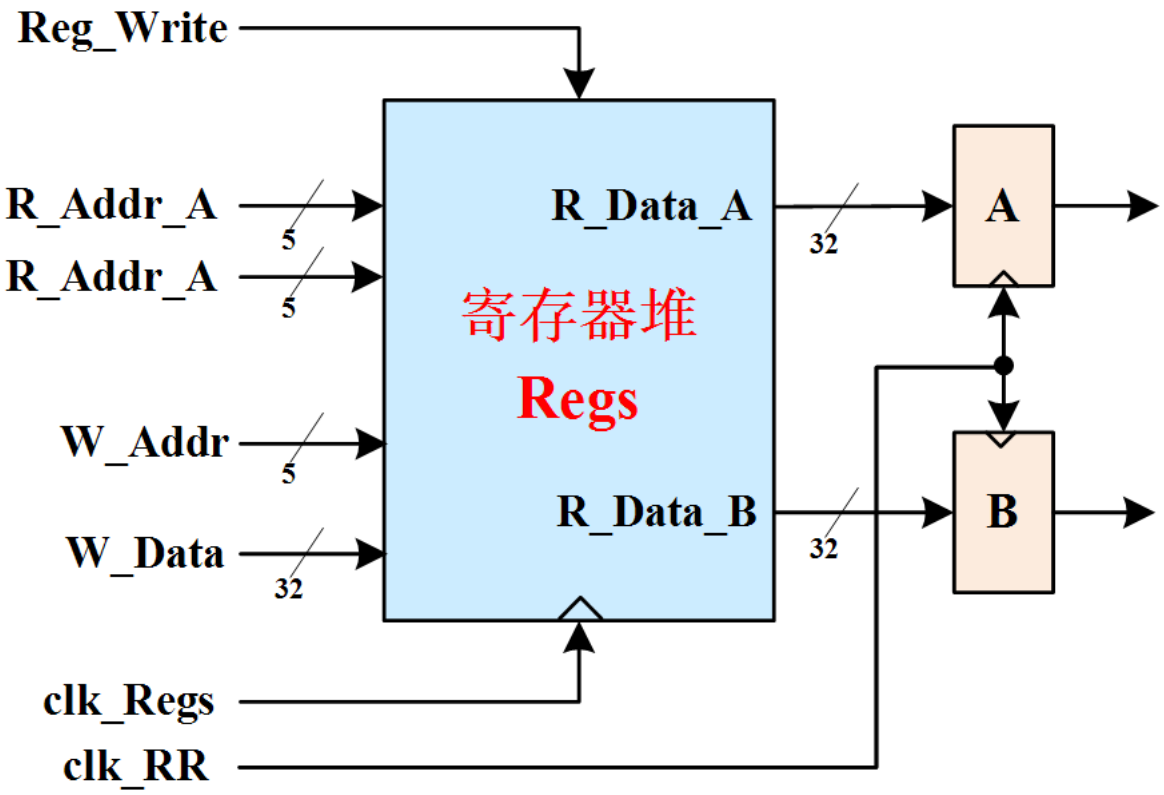
- 运算器部件：
  - （1）通用寄存器堆Regs
  - （2）多功能运算部件ALU
  - （3）标志寄存器FR及标志产生部件FlagU
  - （4）暂存器A、B、F



（1）通用寄存器堆Regs

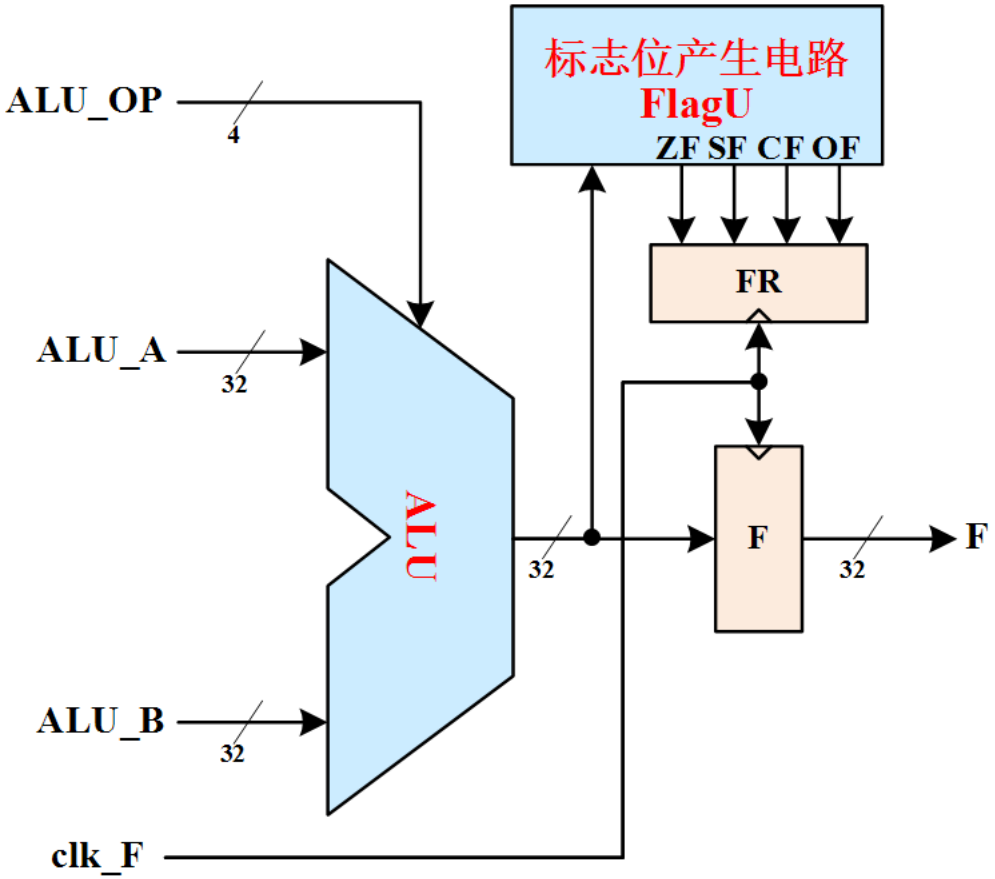
- 32×32位寄存器，寄存器地址5位
  - 三端口：2个读端口和1个写端口
    - 同时读出2个寄存器，写入1个寄存器
    - 读出无须时钟，写入需要时钟

- 读出无须时钟→新问题：多周期CPU，读出的数据存放在哪里？
  - 暂存器A和B
  - 时钟clk_RR：控制打入



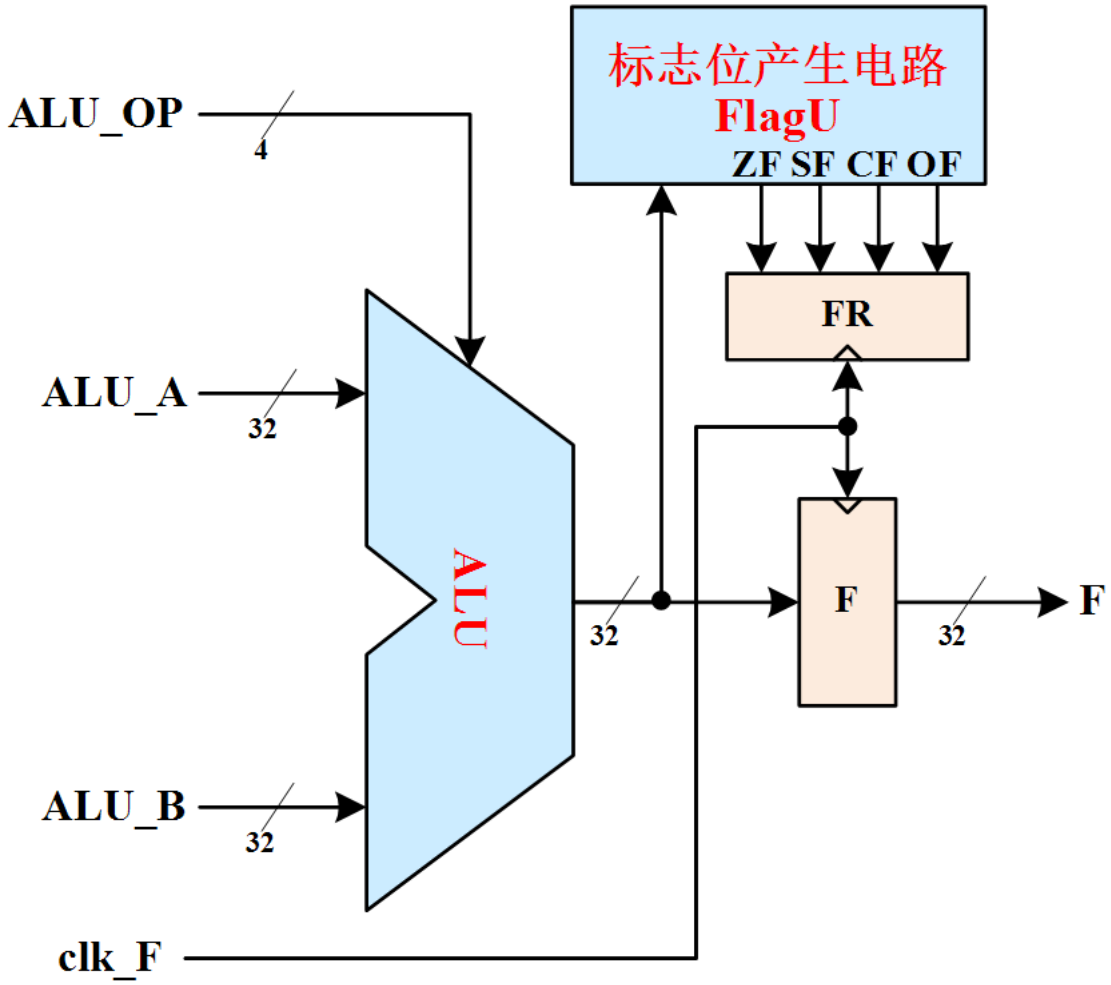
（2）多功能运算部件ALU

- ALU：32位
  - 功能：算术、逻辑、比较置数、移位运算等
  - 选择信号：运算功能码ALU_OP
  - 组合逻辑电路：无须时钟信号
- 同样问题：多周期CPU，运算结果存放在哪里？
  - 暂存器F
  - clk_F：运算结果打入F



（3）标志寄存器FR及标志产生部件FlagU

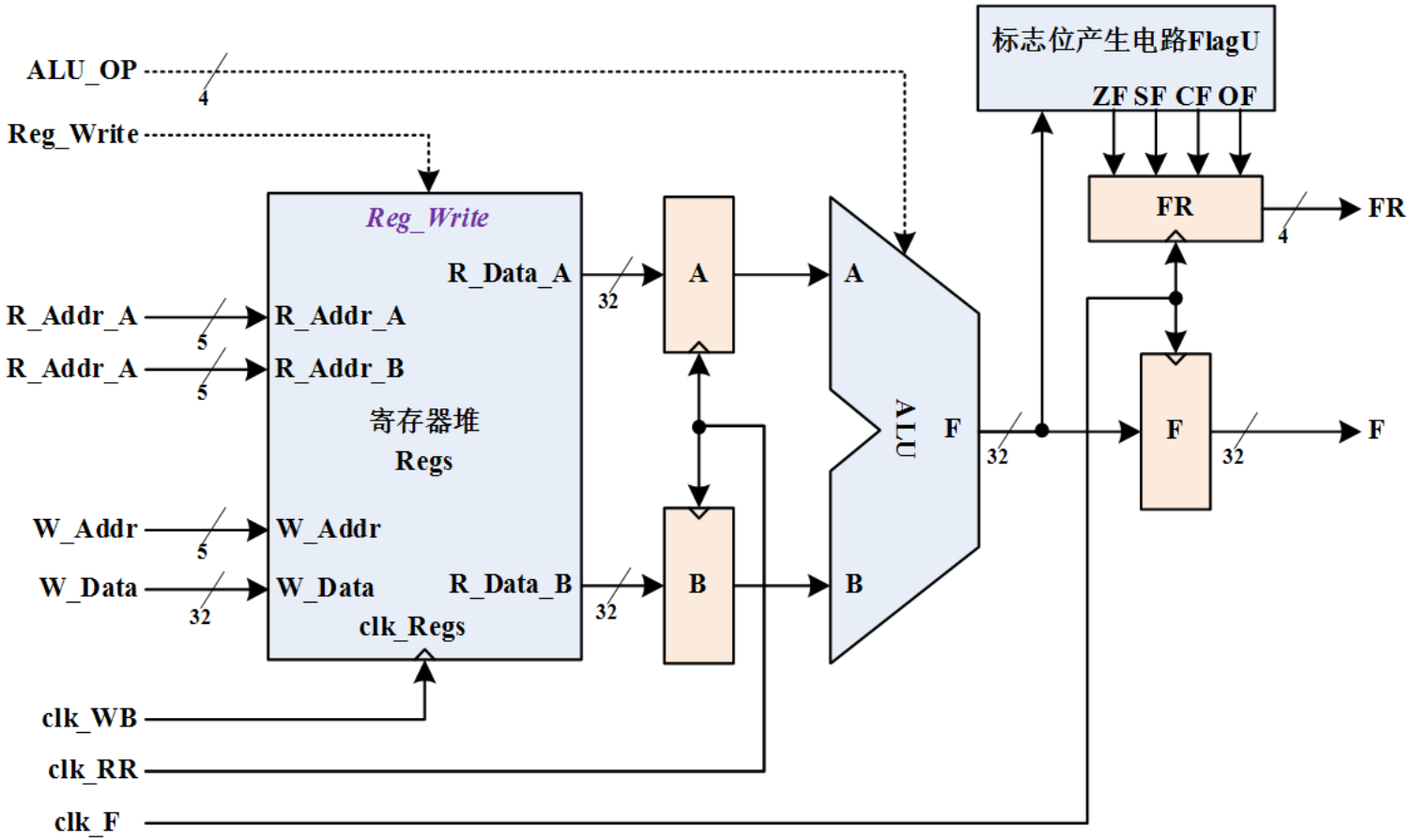
- 标志产生部件FlagU：
  - 组合逻辑电路
  - 输入：运算结果ALU_F
  - 输出： ZF、SF、CF、OF
- 标志寄存器FR：
  - 保存ZF、SF、CF、OF
  - clk_F：ZF、SF、CF、OF 打入FR



（4）暂存器A、B、F

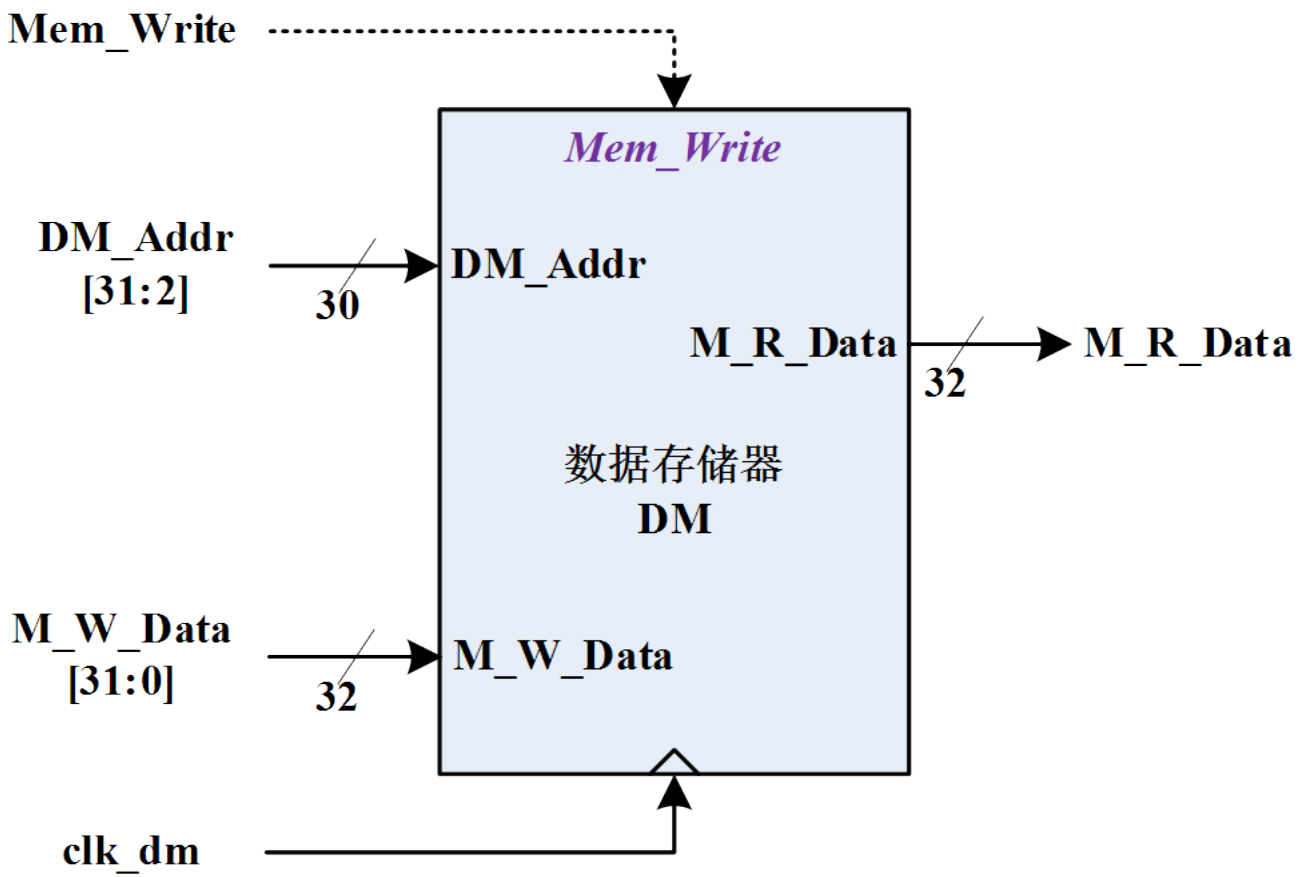
- 32位的寄存器
- 通用寄存器Regs读出的A口数据→A暂存器→ALU的数据输入端A
- 通用寄存器Regs读出的B口数据→B暂存器→ALU的数据输入端B
- ALU的运算结果→F暂存器
- clk_RR：A和B的打入脉冲
- clk_F：F和FR的打入脉冲

- 为什么要设置暂存器?
  - 因为是多周期CPU，一个机器周期操作的结果要存到暂存器中，供下个机器周期操作使用



4、数据存储器DM

- **数据存储器：地址32位，数据32位**
  - 按字节编址，小端模式
  - clk_dm上跳沿：
    - Mem_Write=0：读
    - Mem_Write=1：写
  - DM_Addr：字地址（30位）
  - M_R_Data：读出数据
  - M_W_Data：写入数据



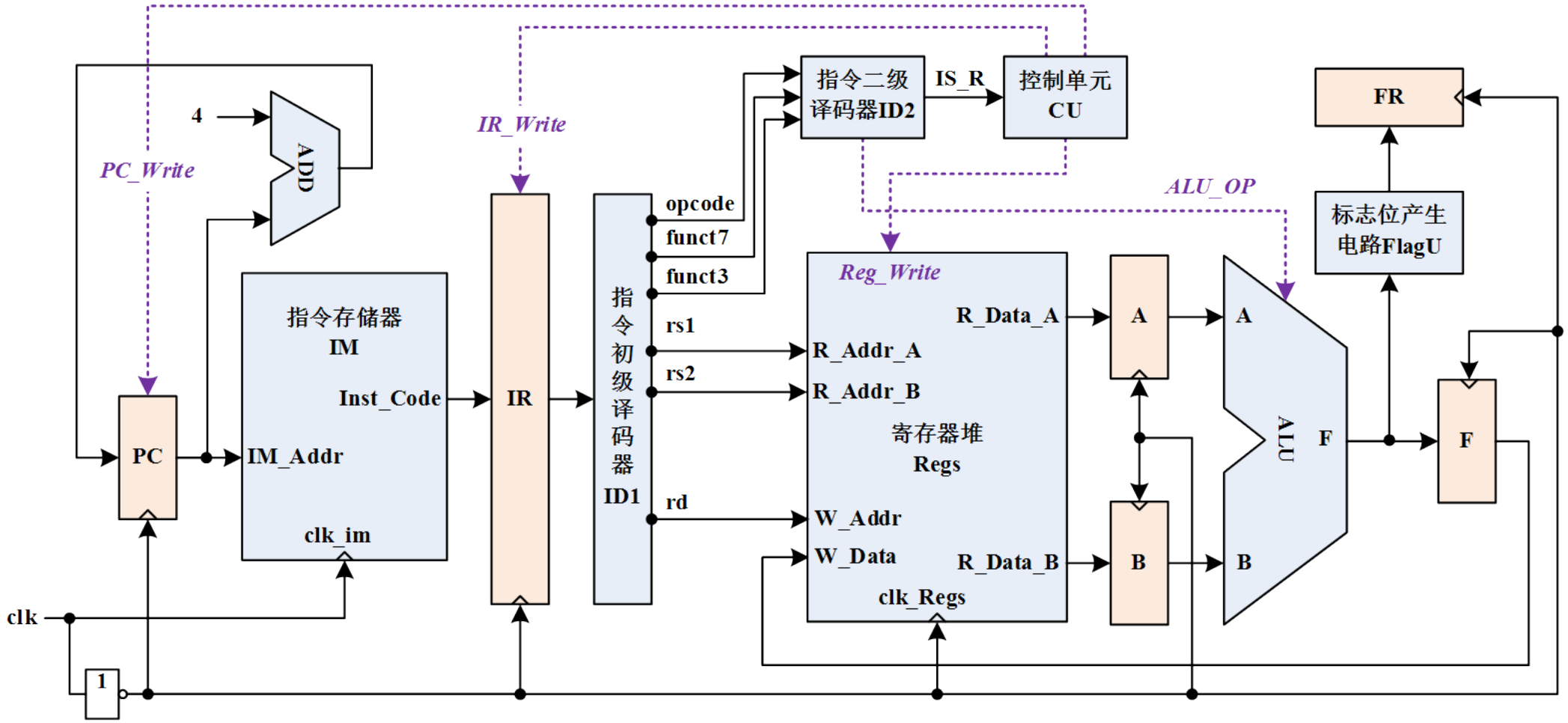
8.1.3 运算及传送指令的数据通路设计

- 1、R型运算指令的数据通路
- 2、I型运算指令的数据通路
- 3、U型传送指令lui的数据通路

- 均从3个方面考虑：
  - 实现指令的系统结构？（如何连接部件？）
  - 指令执行过程？（几个机器周期？）
  - 如何建立数据通路？（发送哪些控制信号？）



❖ 功能：Reg[rs1] op Reg[rs2]→Reg[rd]



①连接取指令和寄存器堆　②连接寄存器堆和ALU部件　③添加译码及控制单元

1、R型运算指令的数据通路部件及连接

- **指令存储器：**
  - 取指令：由PC提供地址，在clk的上跳沿完成读操作；读出数据clk的下跳沿存入IR
- **PC的控制：**
  - 自增：通过专用加法器，+4运算
  - 更新：PC_Write=1，在clk的下降沿完成
- **IR的控制：**
  - 更新： IR_Write=1，在clk的下降沿完成
- **指令初级译码：**
  - 32位指令各字段分别送各数据通路部件
  - rs1、rs2、rd送寄存器堆地址
  - opcode和funct3和funct7送二级译码器ID2译码
- **寄存器堆：**
  - 2个读端口：地址由rs1、rs2指定，读出数据写入暂存器A和B保存
  - 1个写端口：地址由rd指定，写数据从暂存器F的输出端引入
  - 写控制信号：Reg_Write，clk的下跳沿起作用
- **ALU：**
  - 运算功能：10种，由4位ALU_OP指定
  - 输入A和B：由暂存器A和B提供
  - 运算结果输出F：打入暂存器F

1、R型运算指令的数据通路　　控制信号

- R型指令的控制信号：PC_Write,IR_Write,Reg_Write,　ALU_OP
  - PC_Write：在clk的下降沿有效，控制将PC+4打入PC
  - IR_Write：在clk的下降沿有效，控制将指令存储器读出的数据打入IR
  - Reg_Write：在clk的下降沿有效，控制将F写入rd指定的寄存器
  - ALU_OP：由指令译码产生；选择ALU的运算功能

在外面将clk取反，或者这里改成negedge

```
always @(posedge clk or negedge rst_n)
begin
   if(!rst_n)
      PC<= 32'h0000_0000;
   else   begin
      if (PC_Write)
         PC <= PC+4;
   end
end
```

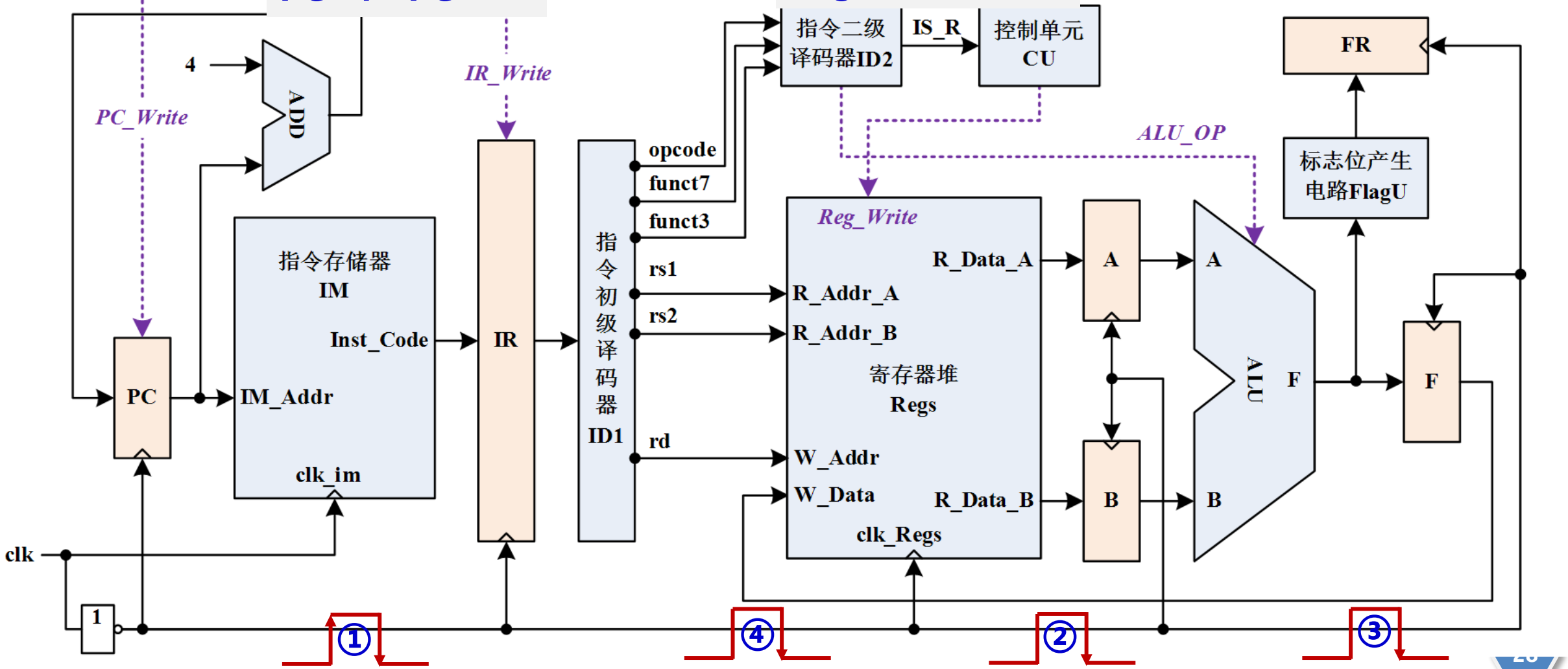
1、R型运算指令的数据通路　　add指令执行过程

①取指令
IMem[PC]→IR
PC+4→PC

②译码及读寄存器
Reg[rs1]→A
Reg[rs2]→B

③运算
A+B→F/FR

④写寄存器
F→Reg[rd]



1、R型运算指令的数据通路　　指令执行过程

❖ R型指令执行过程：

❖ ①IF 取指令，PC自增

- M0：IMem[PC]→IR，PC+4→PC
- 发送控制信号：IR_Write，PC_Write

❖ ②ID 译码及读寄存器

- M1：Reg[rs1]→A，Reg[rs2]→B
- 发送控制信号：无

❖ ③EX ALU运算

- M2：A（op）B→F
- 发送控制信号：ALU_OP

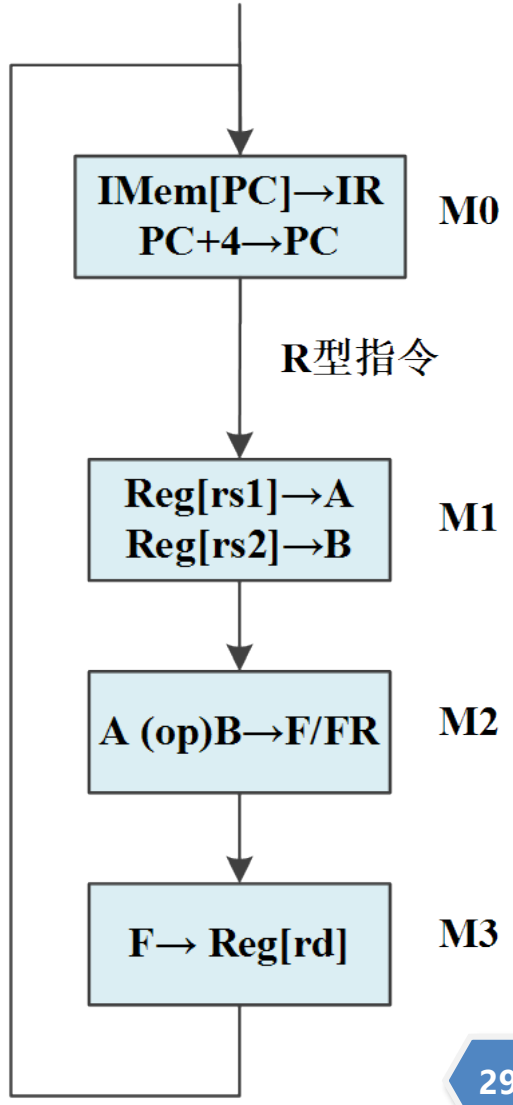
❖ ④WB 写结果寄存器

- M3：F→Reg[rd]
- 发送控制信号：Reg_Write

❖ 暂存器A、B、F没有写使能信号

❖ 每个clk的下降沿，都会执行写入操作

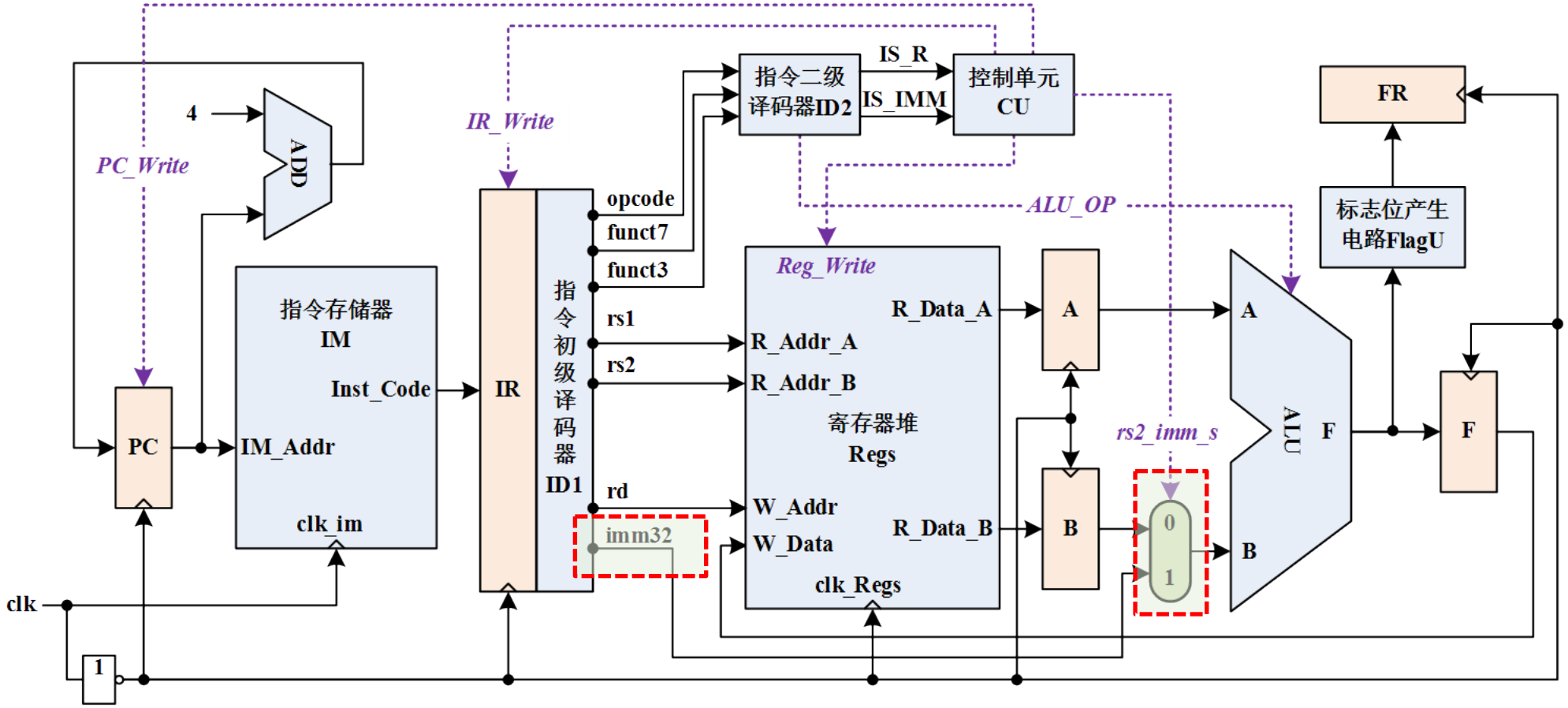
❖ 指令执行的流程图



2、I型运算指令的数据通路　　系统结构、控制信号

❖ 功能：Reg[rs1] op imm32→Reg[rd]，imm32=SE(imm12)

①指令初级译码器产生的imm32要参加运算　　②ALU的B输入端，添加一个二选一数据选择器



新增的控制信号rs2_imm_s

```
assign  ALU_B = rs2_imm_s ? imm32 : B;
```

- R型指令：rs2_imm_s=0，ALU_B=B
- I型指令：rs2_imm_s=1，ALU_B=imm32

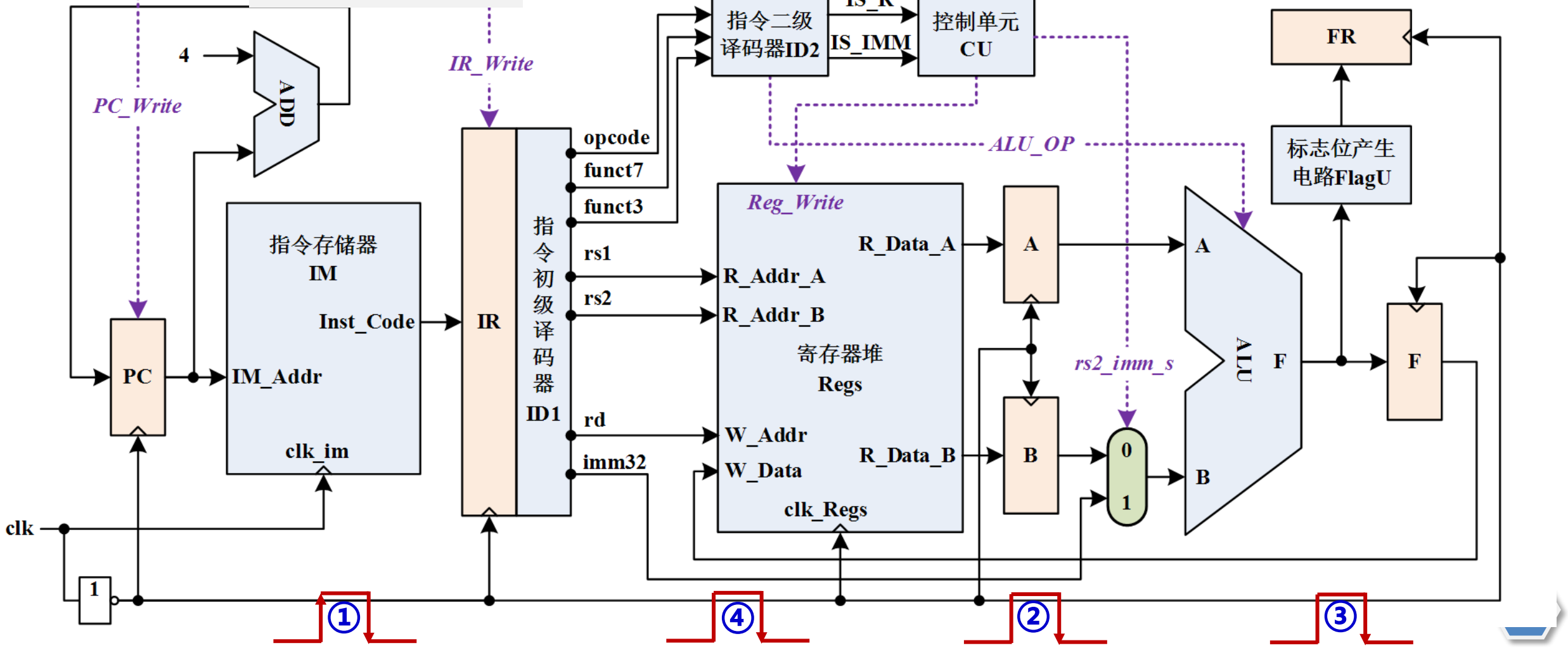
2、I型运算指令的数据通路　　addi指令执行过程

①取指令
IMem[PC]→IR
PC+4→PC

②译码及读寄存器
Reg[rs1]→A

③运算
A+imm32→F/FR

④写寄存器
F→Reg[rd]



2、I型运算指令的数据通路

指令执行过程

❖ I型指令执行过程：

❖ ①IF 取指令，PC自增

- M0：IMem[PC]→IR，PC+4→P

无效
(斜体表示)

❖ ②ID 译码及读寄存器

- M1：Reg[rs1]→A，*Reg[rs2]→B*

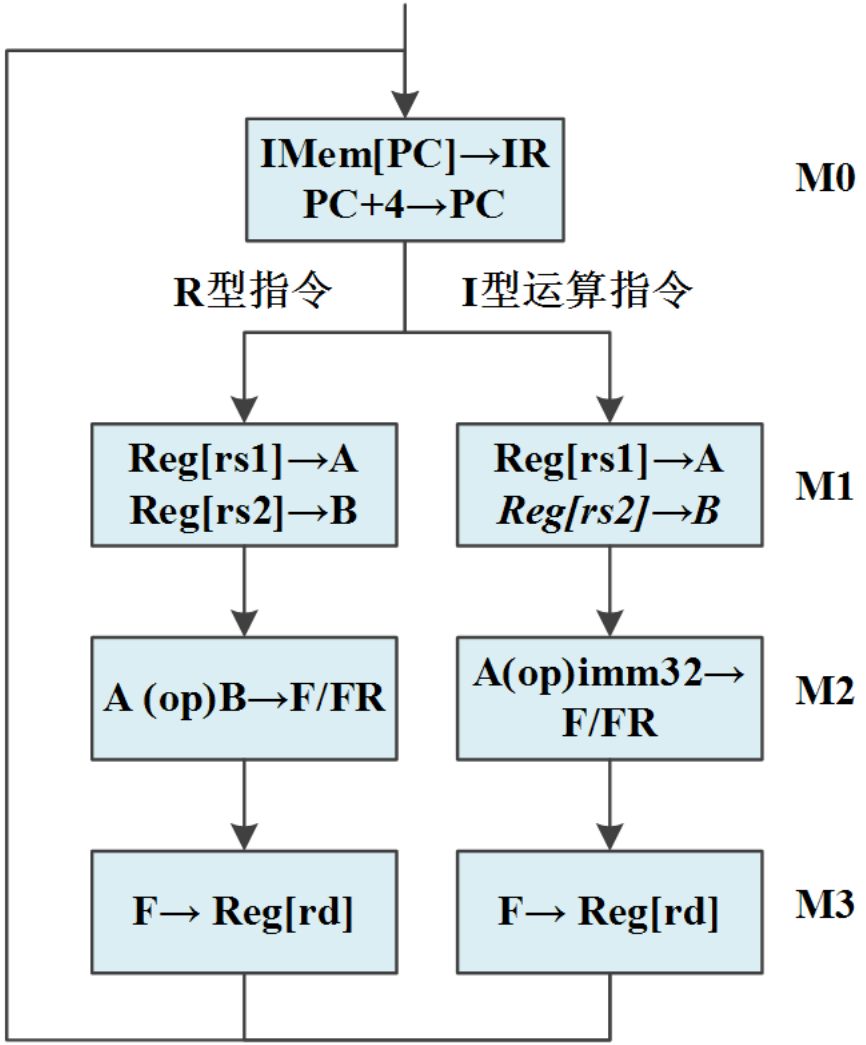
❖ ③EX ALU运算

- M2：A （op）imm32→F

❖ ④WB 写结果寄存器

- M3：F→ Reg[rd]

❖ R-I指令执行的流程图



2、I型运算指令的数据通路 发送的控制信号

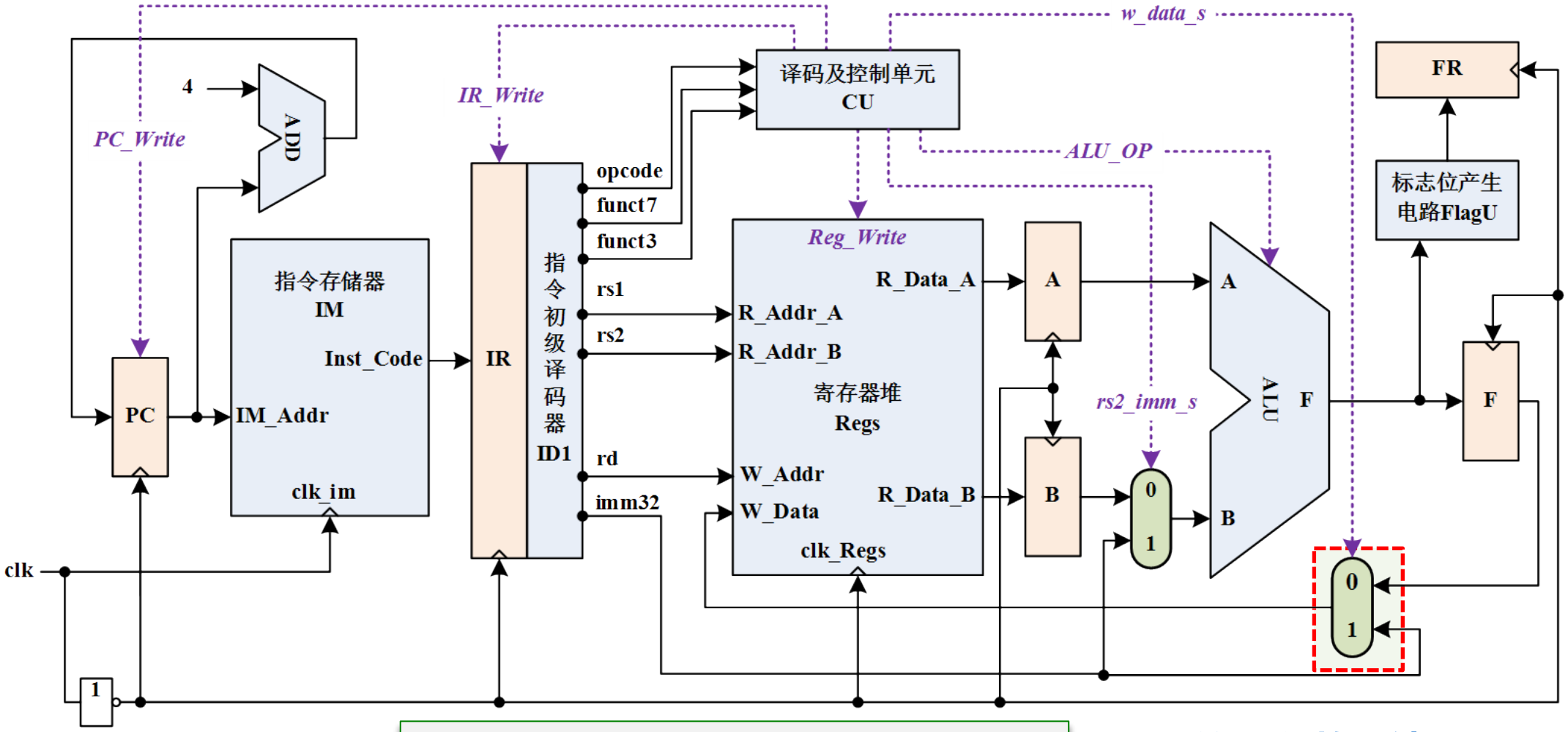
时钟周期	操作	发送控制信号	clk上跳沿	clk下跳沿
M0	IMem[PC]→IR PC+4→PC	IR_Write PC_Write	IMem读操作	打入IR和PC
M1	Reg[rs1]→A *Reg[rs2]→B*	无		打入A和B
M2	A （op） imm32→F/FR	rs2_imm_s=1 ALU_OP=****		打入F和FR
M3	F→ Reg[rd]	Reg_Write		打入rd寄存器

❖R型指令，哪个周期发送的信号不一样？ M2：rs2_imm_s=0



3、U型传送指令lui的数据通路　　系统结构、控制信号

❖ 功能：imm32→Reg[rd]，imm32={imm20,12{0}}



新增控制信号w_data_s

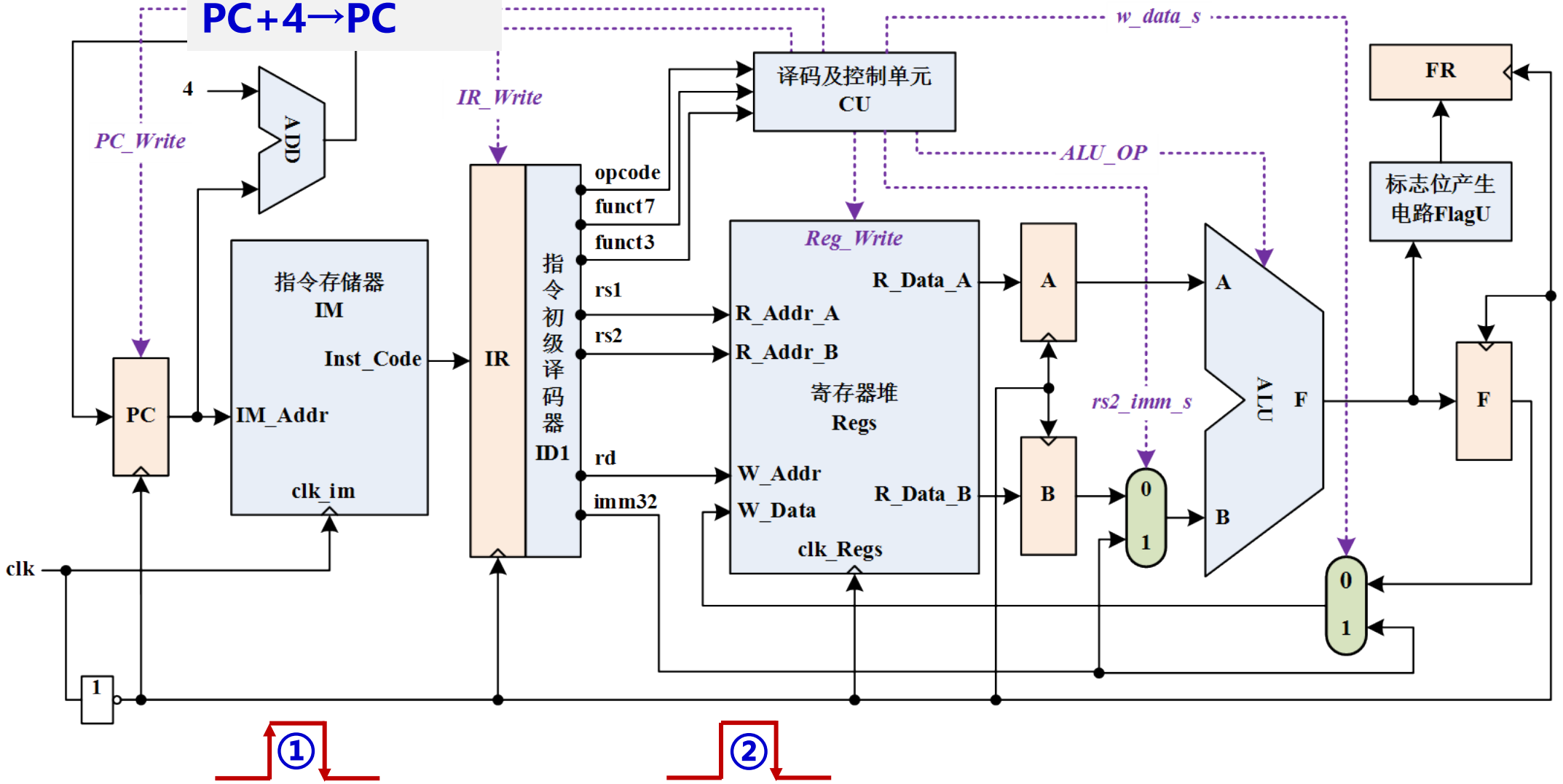
=0：W_Data=F，R型+I型运算指令
=1：W_Data=imm32，lui指令

Regs的写入数据端W_Data
添加二选一数据选择器

3、U型传送指令lui的数据通路　　指令执行过程

①取指令
IMem[PC]→IR
PC+4→PC

②写寄存器
imm32→rd



3、U型传送指令lui的数据通路　　指令执行过程

❖U型lui指令执行过程：

❖①IF 取指令，PC自增

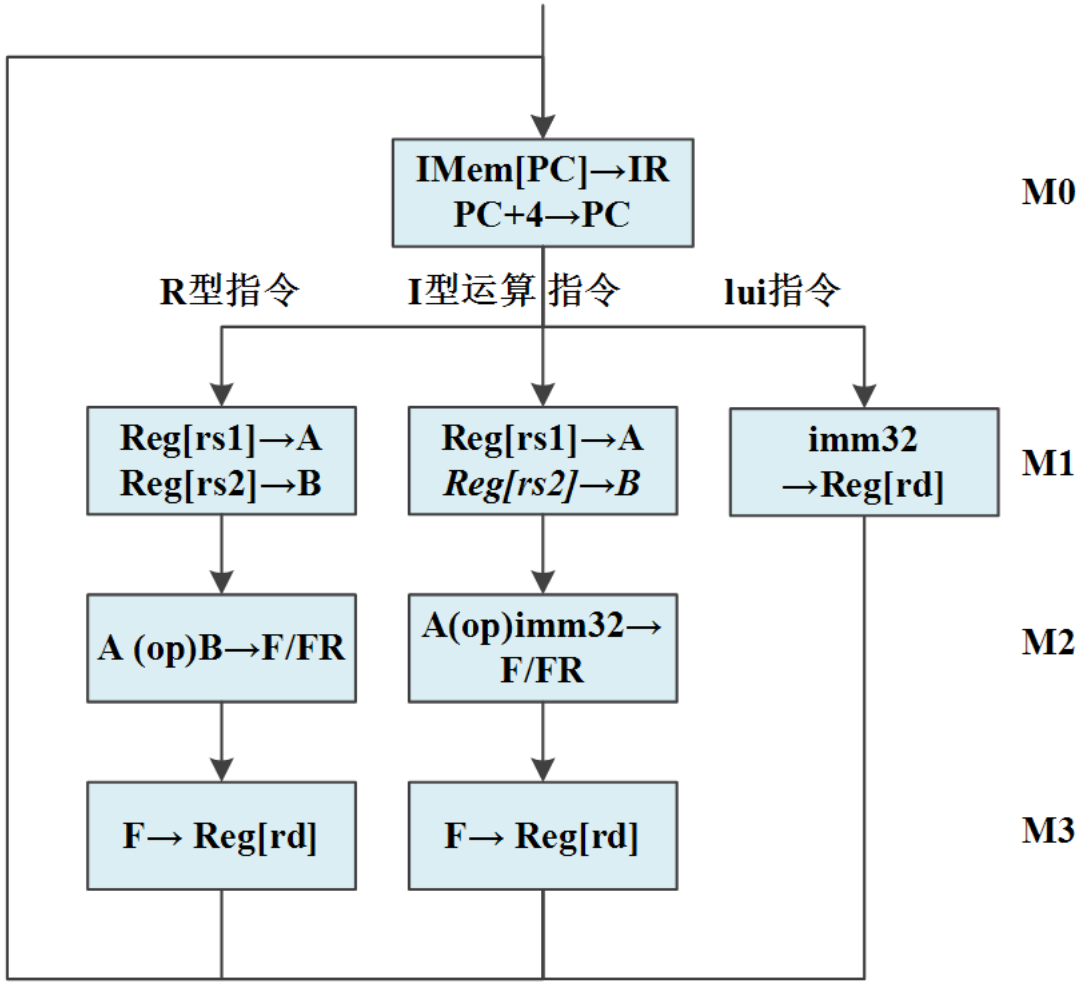
- Mem[PC]→IR，PC+4→PC

❖②WB 写结果寄存器

- imm32→ Reg[rd]

时钟周期	操作	发送控制信号	clk上跳沿	clk下跳沿
M0	IMem[PC]→IR,PC+4→PC	IR_Write PC_Write	IMem读操作	打入IR和PC
M1	imm32→Reg[rd]	w_data_s=1 Reg_Write		打入rd

❖R-I-U指令执行的流程图



R型+I型运算指令+U型指令lui　　发送的控制信号

指令	机器周期	执行操作	发送的控制信号
所有指令	M_0	IMem[PC]→IR，PC+4→PC	IR_Write，PC_Write
R型指令	IS_R·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_R·M_2	A（op）B→F	ALU_OP=****，rs1_imm_s=0
	IS_R·M_3	F→Reg[rd]	Reg_Write，w_data_s=0
I型运算指令	IS_IMM·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_IMM·M_2	A（op）imm32→F	ALU_OP=****，rs1_imm_s=1
	IS_IMM·M_3	F→Reg[rd]	Reg_Write，w_data_s=0
lui指令	IS_LUI·M_1	imm32→Reg[rd]	Reg_Write，w_data_s=1

8.1.4 访存指令的数据通路设计 指令分析

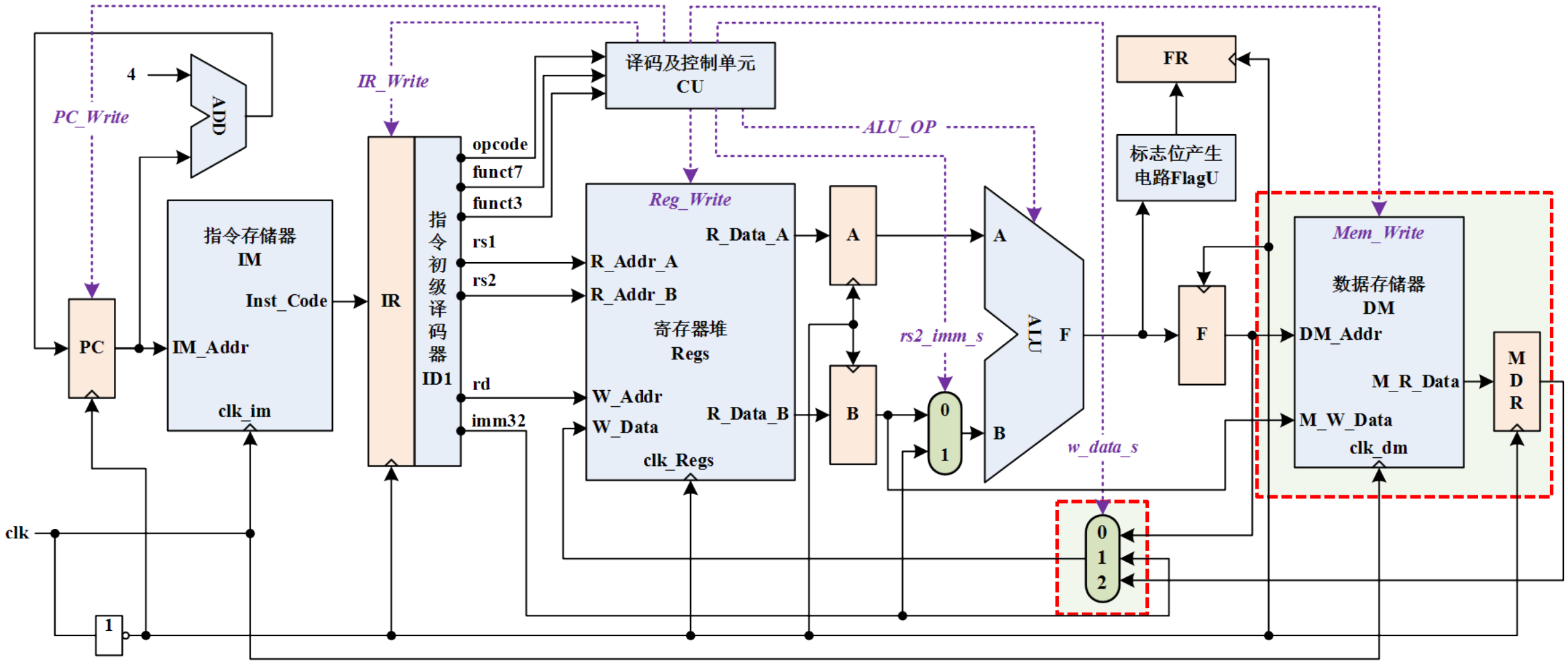
- 取数指令lw：I型格式
  - 功能：DMem[Reg[rs1]+imm32]→Reg[rd]
- 存数指令sw：S型格式
  - 功能：Reg[rs2]→DMem[Reg[rs1]+imm32]
- 需要添加：数据存储器部件DM
  - 如何连接?
  - EA=Reg[rs1]+imm32：**谁完成计算?**
  - imm32=SE32(imm12)

8.1.4 访存指令的数据通路设计 数据通路

- ①访问DM时，地址DM_Addr从哪里来?
  - EA=Reg[rs1]+imm32，如何实现地址加法?
  - Reg[rs1]→A，rs2_imm_s=1，则A+imm32→F
  - 故：访问DM时，地址DM_Addr可以直接由暂存器F提供
- ②执行DM读操作时，读出的数据M_R_Data到哪里去?
  - lw指令：执行DM读操作，读出数据要写入rd寄存器
  - 存储器读出数据端：设置一个暂存器MDR，保存存储器读出的数据
  - 在下一个周期，再将MDR的内容写回到rd寄存器
  - 故：执行DM读操作时，读出的数据M_R_Data保存到MDR寄存器。
  - 写回周期：写入rd的写数据W_Data来源，多了MDR，故W_Data的二选一→三选一
- ③执行DM写操作时，写入的数据M_W_Data由谁提供?
  - sw指令：执行数据存储器的写操作，写入的数据是rs2寄存器的内容
  - rs2寄存器内容：保存在B暂存器中；
  - 故：执行DM写操作时，写入的数据M_W_Data由暂存器B提供

8.1.4 访存指令的数据通路设计 系统结构

①添加数据存储器DM ②添加MDR ③寄存器堆的写入数据多路选择器→三选一



w_data_s=0: W_Data=F
w_data_s=1: W_Data=imm32
w_data_s=2: W_Data=MDR

8.1.4 访存指令的数据通路设计 部件、控制信号

EA=rs1+imm32
ALU结果存入→F

❖数据存储器：

- 地址：F暂存器提供，Why?
- 读出数据：接暂存器MDR，clk上跳沿读出，下跳沿打入MDR
- 写入数据：从暂存器B引入，Why?
- 写使能信号：Mem_Write=1，clk上跳沿写入。

sw：rs2→Mem，
rs2内容在B中

❖W_Data的三选一数据选择器

- R型指令：w_data_s=0：W_Data=F
- I型运算指令：w_data_s=0：W_Data=F
- U型指令lui：w_data_s=1：W_Data=imm32
- I型访存指令lw： w_data_s=2：W_Data=MDR

❖MDR

- 输入：DMem读出的数据
- clk下跳沿打入
- 输出：送W_Data

8.1.4 访存指令的数据通路设计

指令执行过程

lw

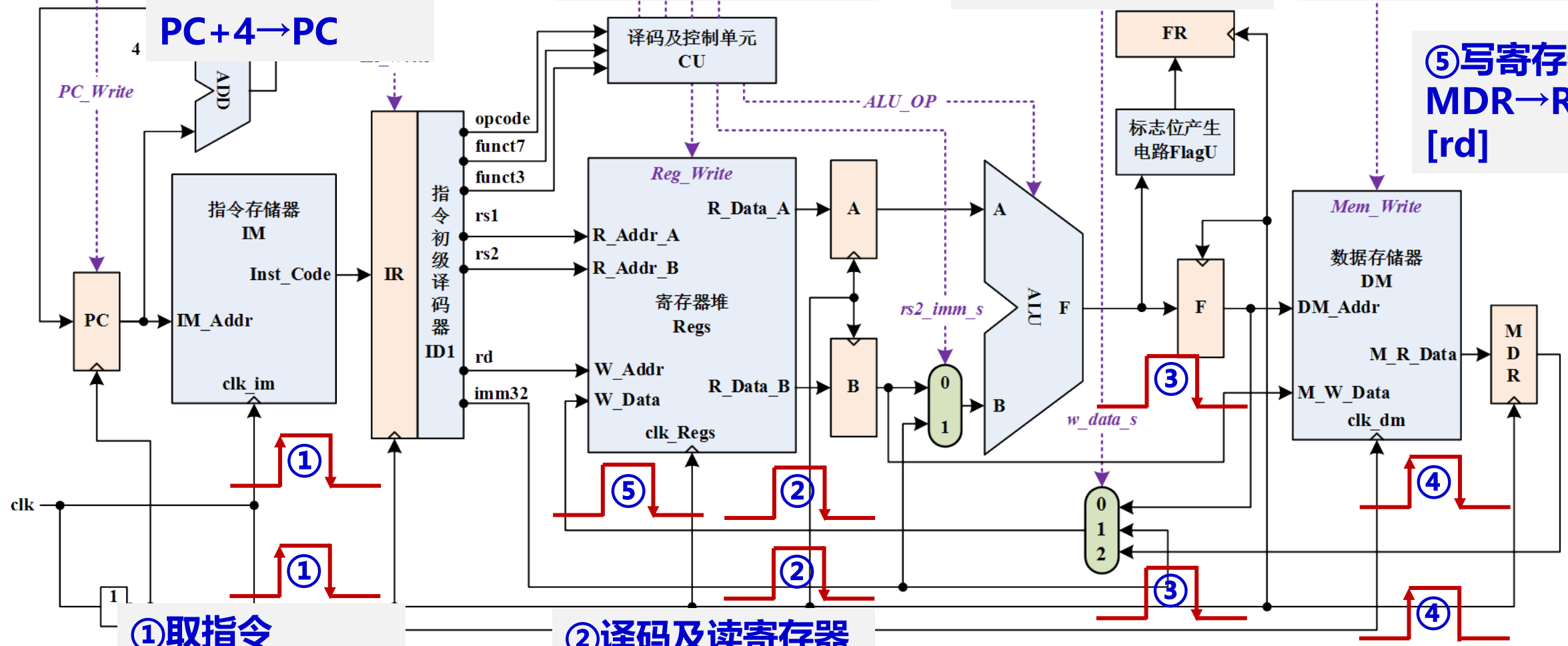
①取指令
IMem[PC]→IR
PC+4→PC

②译码及读寄存器
Reg[rs1]→A

③计算EA
A+imm32→F/FR

④读存储器
Mem32[F]→MDR

⑤写寄存器
MDR→Reg[rd]



sw

①取指令
IMem[PC]→IR
PC+4→PC

②译码及读寄存器
Reg[rs1]→A
Reg[rs2]→B

③计算EA
A+imm32→F/FR

④写存储器
B→Mem32[F]

❖I型lw指令执行过程：

❖①IF 取指令，PC自增

- Mem[PC]→IR，PC+4→PC

❖②ID 译码及读寄存器

- Reg[rs1]→A，*Reg[rs2]→B*

❖③EX ALU计算EA

- A +imm32→F

❖④Mem 读存储器

- Mem32[F]→MDR

❖⑤WB 写结果寄存器

- MDR→ Reg[rd]

❖S型sw指令执行过程：

❖①IF 取指令，PC自增

- Mem[PC]→IR，PC+4→PC

❖②ID 译码及读寄存器

- Reg[rs1]→A，Reg[rs2]→B

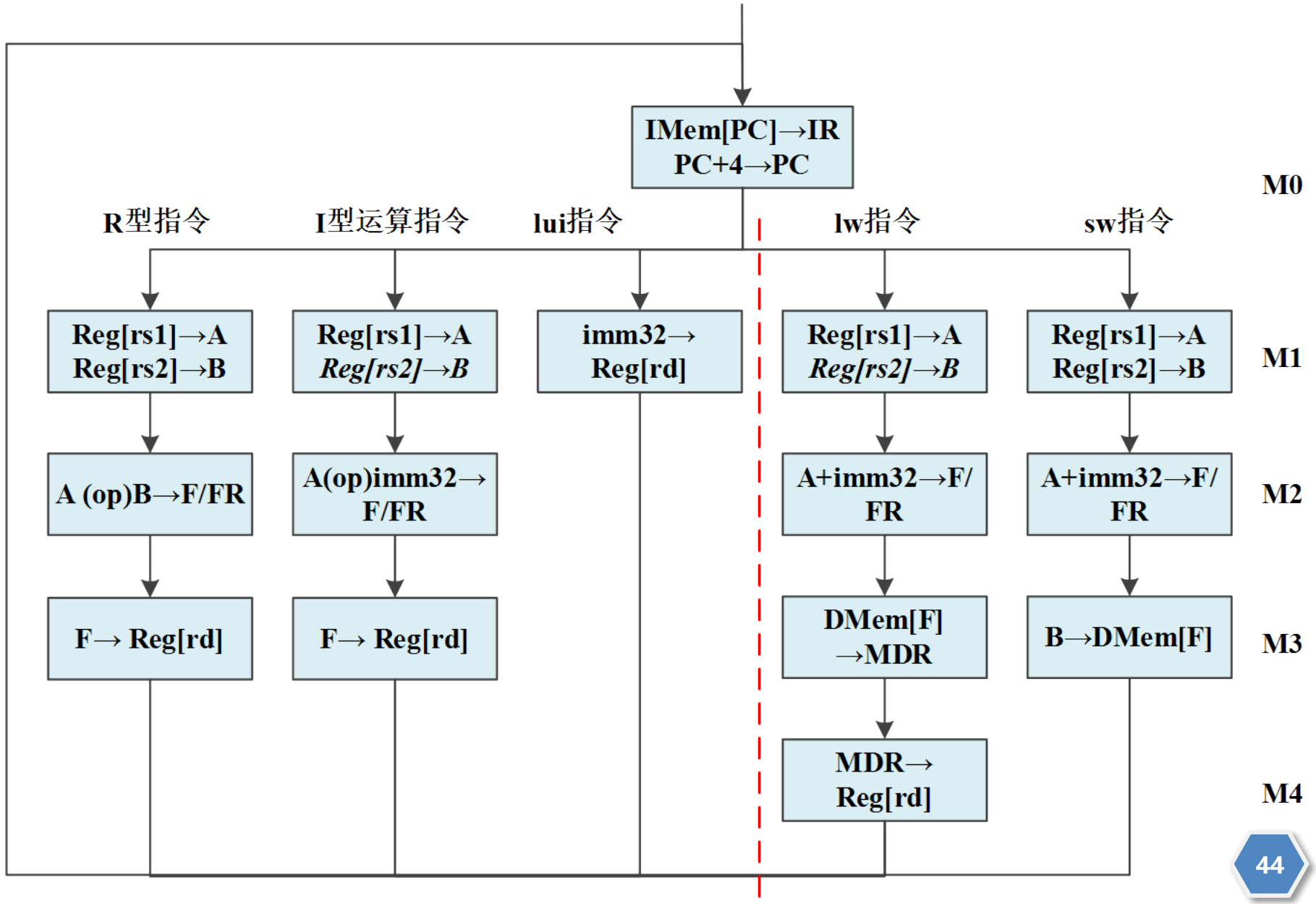
❖③EX ALU计算EA

- A +imm32→F

❖④Mem 写存储器

- B→Mem32[F]

❖R-I-U-S指令执行的流程图



8.1.4 访存指令的数据通路设计 发送的控制信号

时钟周期	lw指令操作	发送控制信号	clk上跳沿	clk下跳沿
M0	IMem[PC]→IR,PC+4→PC	IR_Write，PC_Write	IMem读操作	打入IR和PC
M1	Reg[rs1]→A,*Reg[rs2]→B*	无		打入A和B
M2	A + imm32→F/FR	rs2_imm_s=1,ALU_OP=0000		打入F
M3	DMem[F]→MDR	无	DMem读操作	打入MDR
M4	MDR→ Reg[rd]	w_data_s=2,Reg_Write		打入rd

时钟周期	sw指令操作	发送控制信号	clk上跳沿	clk下跳沿
M0	IMem[PC]→IR,PC+4→PC	IR_Write，PC_Write	IMem读操作	打入IR和PC
M1	Reg[rs1]→A,Reg[rs2]→B	无		打入A和B
M2	A + imm32→F/FR	rs2_imm_s=1,ALU_OP=0000		打入F
M3	B→DMem[F]	Mem_Write	DMem写操作	

R型+I型运算+lui+I型lw+S型sw指令 发送的控制信号

指令	机器周期	执行操作	发送的控制信号
所有指令	M_0	IMem[PC]→IR，PC+4→PC	IR_Write，PC_Write
R型 运算指令	IS_R·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_R·M_2	A（op）B→F	ALU_OP，rs2_imm_s=0
	IS_R·M_3	F→Reg[rd]	Reg_Write，w_data_s=00
I型 运算指令	IS_IMM·M_1	Reg[rs1]→A，*Reg[rs2]→B*	无
	IS_IMM·M_2	A（op）imm32→F	ALU_OP，rs2_imm_s=1
	IS_IMM·M_3	F→Reg[rd]	Reg_Write，w_data_s=00
lui指令-U型	IS_LUI·M_1	imm32→Reg[rd]	Reg_Write，w_data_s=01
lw指令-I型	IS_LW·M_1	Reg[rs1]→A，*Reg[rs2]→B*	无
	IS_LW·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_LW·M_3	DMem[F]→MDR	无
	IS_LW·M_4	MDR→Reg[rd]	Reg_Write，w_data_s=10
sw指令-S型	IS_SW·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_SW·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_SW·M_3	B→DMem[F]	Mem_Write



8.1.5 转移类指令的数据通路设计 指令分析

❖指令格式与编码： B型指令beq+J型指令jal+I型指令jalr

位数	7位	5位	5位	3位	5位	7位	指令功能
B型指令字段	imm[12,10:5]	rs2	rs1	funct3	imm[4:1,11]	opcode	
beq rs1,rs2,offset12	offset7	rs2	rs1	000	offset5	1100011	if(rs1=rs2)PC+SE32(offset12*2)→PC（相等则跳转）

位数	20位	5位	7位	指令功能
J型指令字段	imm[20,10:1,11,19:12]	rd	opcode	
jal rd,offset20	offset20（=offset[20:1])	rd	1101111	PC+4→rd,PC+SE32(offset20*2)→PC（相对跳转并链接）

位数	12位	5位	3位	5位	7位	指令功能
I型指令字段	imm[11:0]	rs1	funct3	rd	opcode	
jalr rd,offset12(rs1)	offset12	rs1	000	rd	1100111	PC+4→rd,rs1+SE32(offset12)→PC（寄存器相对跳转并链接）

8.1.5 转移类指令的数据通路设计　　指令分析

- 分析指令功能，需要添加什么新的部件?
- 转移指令的根本特征：修改PC的值；PC的来源多了：
  - B型beq：PC+SE32(offset12*2)→PC
  - J型jal：　PC+SE32(offset20*2)→PC
  - I型jalr：　rs1+SE32(offset12)→PC

PC+imm32→PC

rs1+imm32→PC

- 两个问题：
- ①根据哪个来源更新PC的值?
  - 设置多路选择器，选择一个，更新PC的值
  - 三选一：PC+4/ PC+imm32/ rs1+imm32
- ②怎么产生新的PC值?
  - rs1+imm32：可利用ALU运算
  - PC+imm32：用ALU？还是独立设置一个加法器?
    - 独立设置一个加法器
    - 为什么不用ALU?

8.1.5 转移类指令的数据通路设计　　指令分析

指令	格式	功能	执行过程			
beq rs1,rs2, offset12	B型	if(rs1=rs2)PC+SE32 (offset12*2)→PC	取指令	读寄存器 rs1、rs2	ALU做减法置ZF	ZF=1则转移地址写PC
jal rd,offset20	J型	PC+4→rd,PC+SE32 (offset20*2)→PC	取指令	写寄存器rd 转移地址写PC		
jalr rd, offset12(rs1)	I型	PC+4→rd,rs1+SE32 (offset12)→PC	取指令	读寄存器 rs1	ALU计算转移地址	写寄存器rd，写PC

❖ rd多一个写入选项：

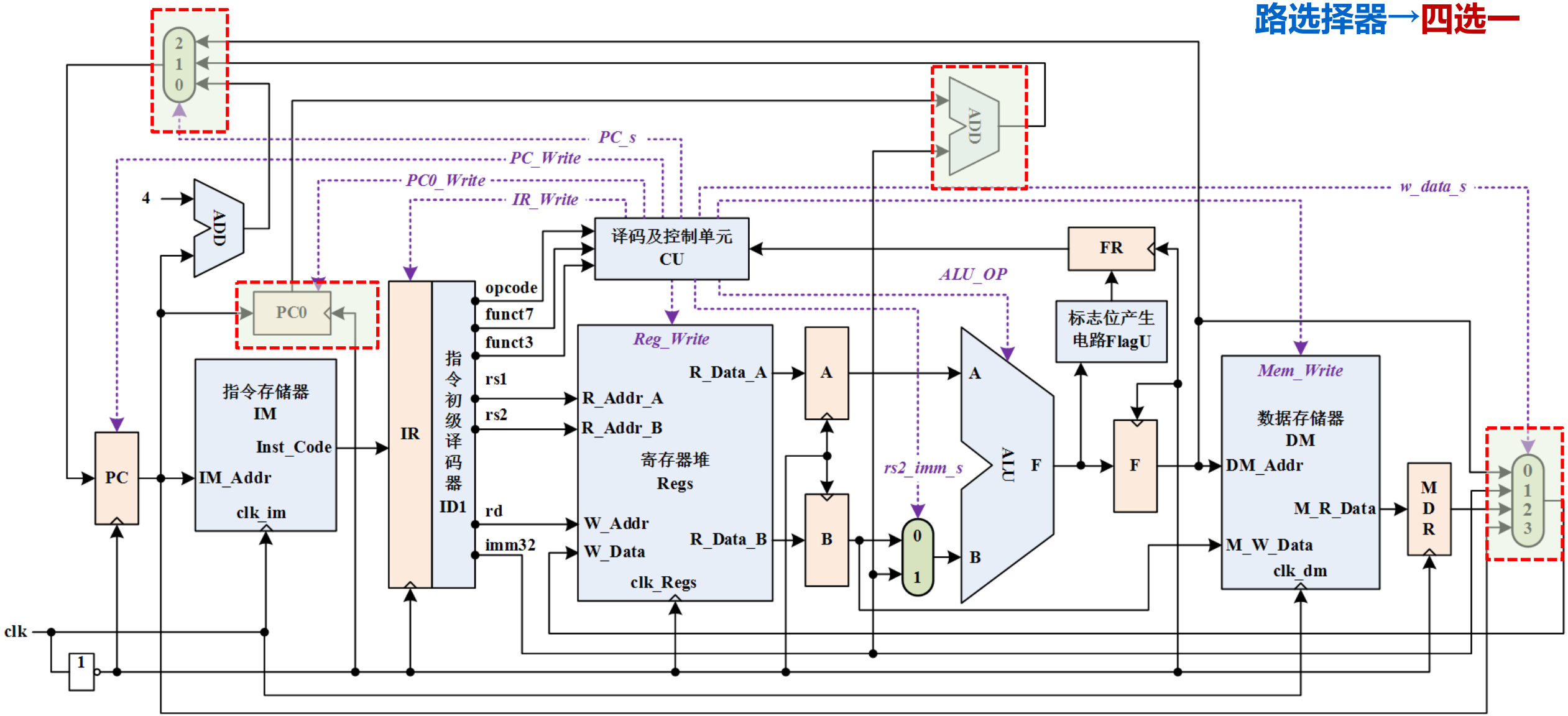
- PC+4→rd
- 在执行这步操作时，PC已经+4
- 故：应执行PC→rd

❖ 转移地址专用加法器：

- PC+imm32：PC是+4前的PC
- 在执行这步操作时，PC已经+4
- 故：应添加一个PC0，存储+4前的PC

8.1.5 转移类指令的数据通路设计　　系统结构

①添加PC0　②添加相对转移地址加法器　③添加PC写入多路选择器　④修改rd写入数据多路选择器→四选一



8.1.5 转移类指令的数据通路设计 部件、控制信号

❖转移地址专用加法器

- 输入：PC0+imm32
- 输出：运算结果送入 PC写入多路选择器

❖PC0暂存器

- 输入：PC，取指令的周期，在clk下跳沿，PC0_Write=1，将PC打入PC0
- 输出：送入转移地址专用加法器

❖PC写入多路选择器：三选一

- 所有指令的取指令周期：PC_s=0：PC+4→PC
- beq分支指令：ZF=1则PC_s=1： PC0+imm32→PC
- jal转移并链接指令：PC_s=1： PC0+imm32→PC
- jalr转移并链接指令： PC_s=2： F（=rs1+imm32）→PC

8.1.5 转移类指令的数据通路设计 部件、控制信号

- W_Data的四选一数据选择器
  - R型指令：w_data_s=0：W_Data=F
  - I型运算指令：w_data_s=0：W_Data=F
  - U型指令lui：w_data_s=1：W_Data=imm32
  - I型访存指令lw： w_data_s=2：W_Data=MDR
  - J型转移指令jal：w_data_s=3：W_Data=PC
  - I型转移指令jalr：w_data_s=3：W_Data=PC

- B型beq指令呢？ w_data_s=?
- S型sw指令呢?

- 无所谓，因为不写rd，Reg_Write=0

8.1.5 转移类指令的数据通路设计 指令执行过程

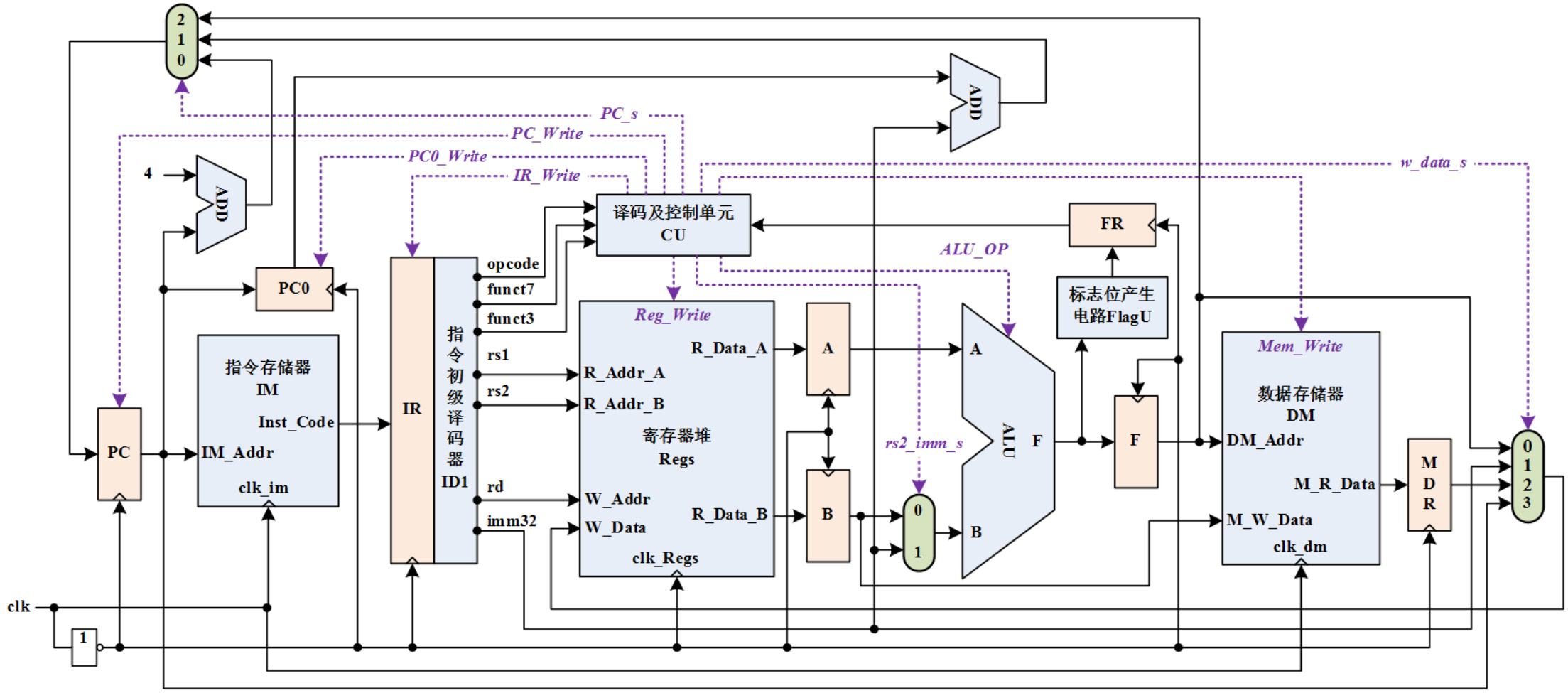
beq

①取指令
IMem[PC]→IR
PC+4→PC
PC→PC0

②读寄存器
Reg[rs1]→A
Reg[rs2]→B

③减法计算
A-B→F/FR

④分支
ZF=1:PC0+imm32→PC
ZF=0：空



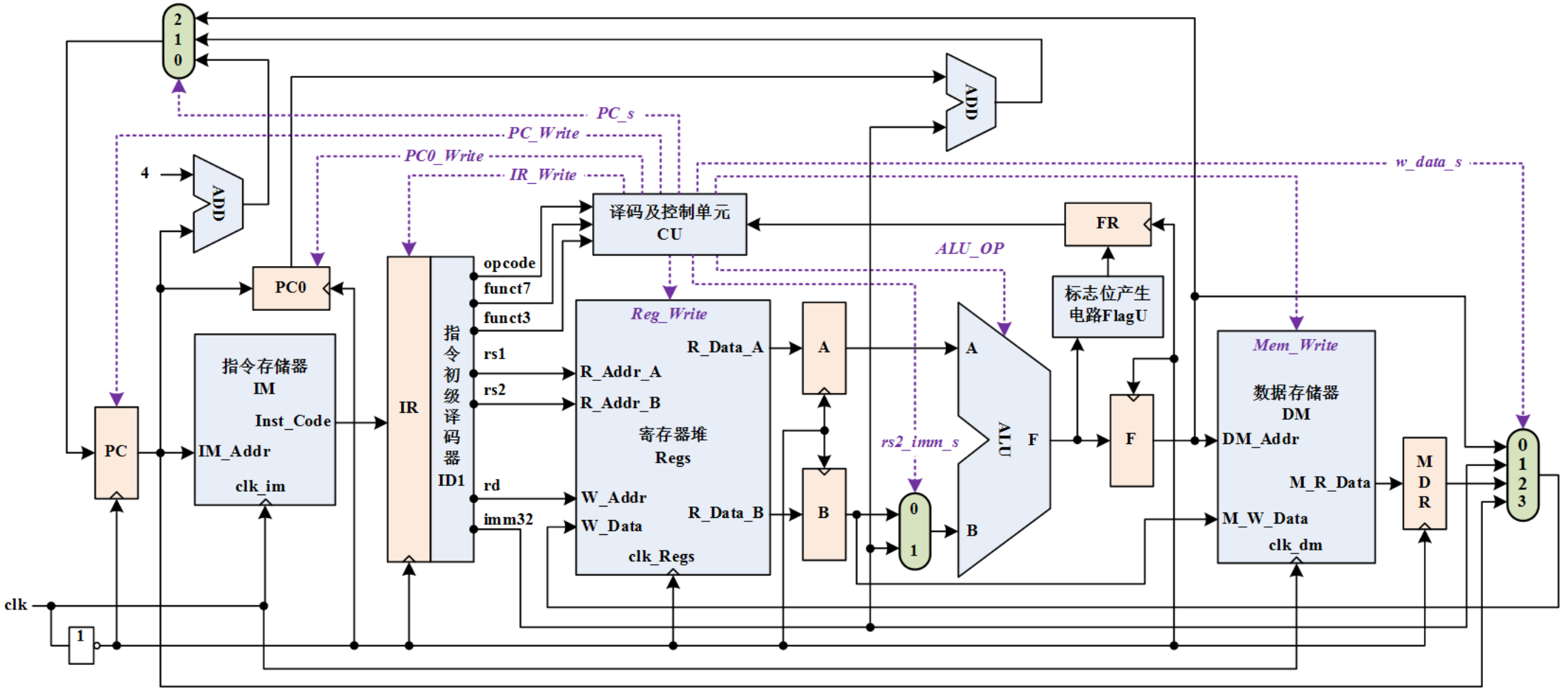
8.1.5 转移类指令的数据通路设计

指令执行过程

jal

①取指令
IMem[PC]→IR
PC+4→PC
PC→PC0

②写寄存器 PC→Reg[rd]
转移PC0+imm32→PC



8.1.5 转移类指令的数据通路设计

指令执行过程

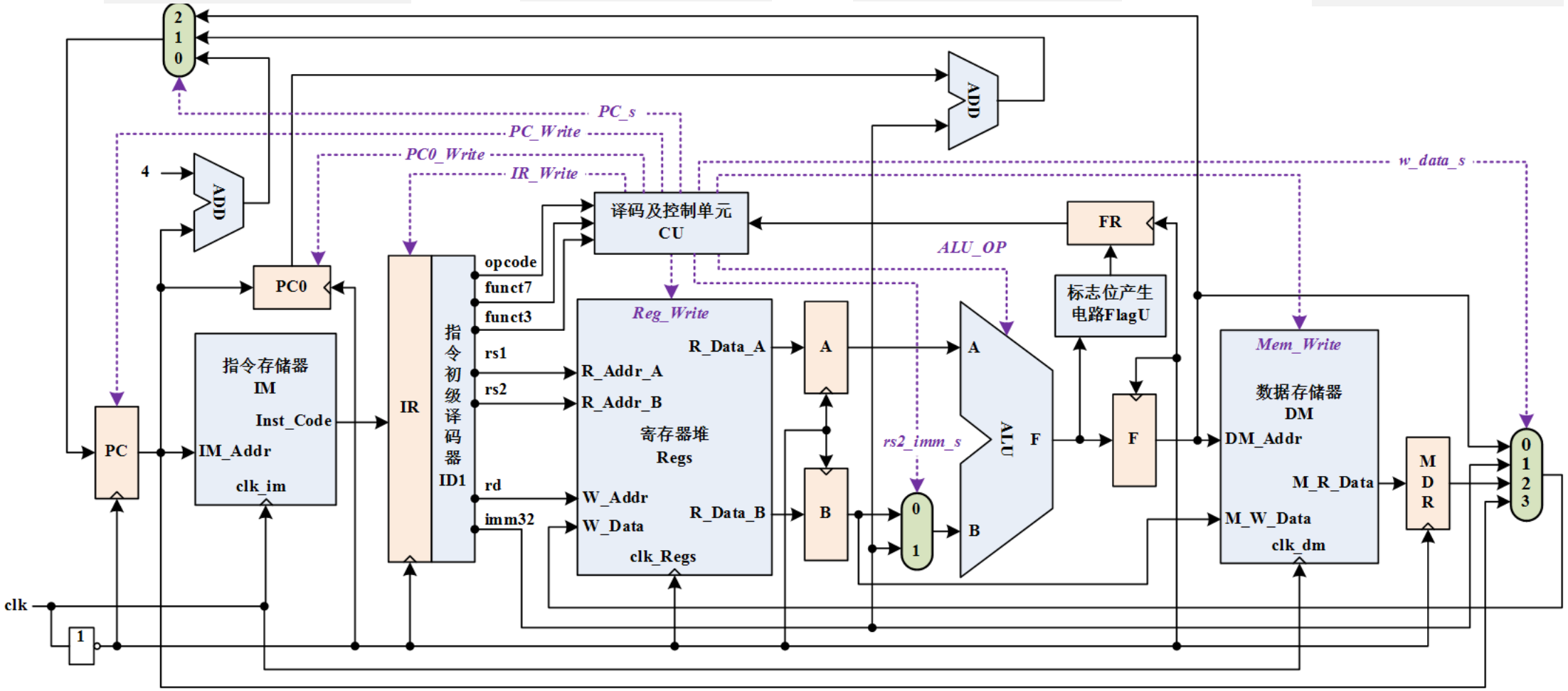
jalr

①取指令
IMem[PC]→IR
PC+4→PC
PC→PC0

②读寄存器
Reg[rs1]→A

③计算转移地址
A+imm32→F

④写寄存器
PC→Reg[rd]
转移：F→PC



8.1.5 转移类指令的数据通路设计 发送的控制信号

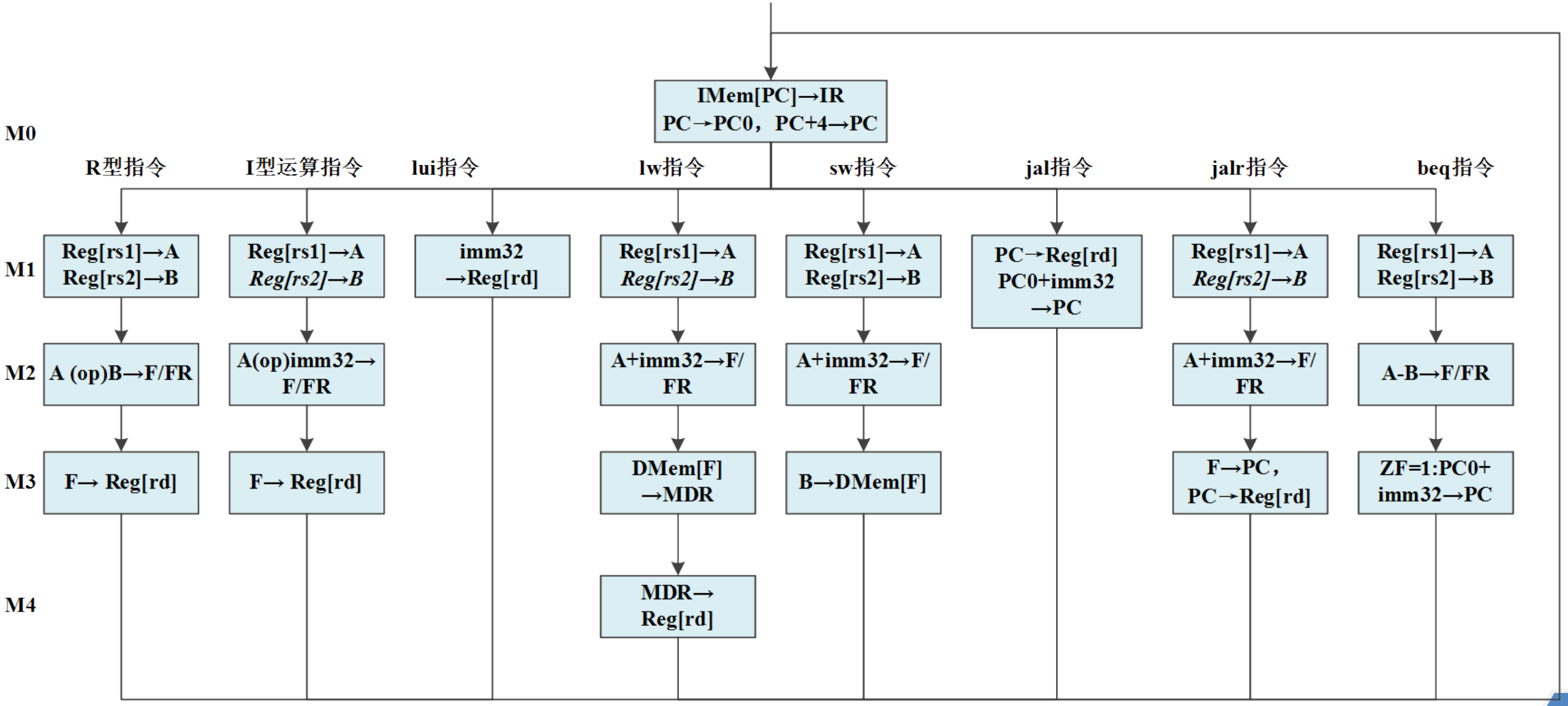
时钟周期	beq指令操作	发送控制信号	clk上跳沿	clk下跳沿
M0	IMem[PC]→IR,PC+4→PC PC→PC0	IR_Write,PC_Write, PC0_Write,PC_s=00	IMem 读操作	打入IR、PC、PC0
M1	Reg[rs1]→A,Reg[rs2]→B	无		打入A和B
M2	A -B→F/FR	rs2_imm_s=0,ALU_OP=1000		打入F、FR
M3	ZF=1:PC0+imm32→PC	ZF=1：PC_s=01,PC_Write		ZF=1则打入PC

时钟周期	jal指令操作	发送控制信号	clk上跳沿	clk下跳沿
M0	IMem[PC]→IR,PC+4→PC PC→PC0	IR_Write,PC_Write, PC0_Write,PC_s=00	IMem 读操作	打入IR、PC、PC0
M1	PC→Reg[rd] PC0+imm32→PC	w_data_s=3，Reg_Write PC_s=01，PC_Write		打入rd和PC

时钟周期	jalr指令操作	发送控制信号	clk上跳沿	clk下跳沿
M0	IMem[PC]→IR,PC+4→PC PC→PC0	IR_Write，PC_Write，PC0_Write， PC_s=00	IMem 读操作	打入IR、PC、PC0
M1	Reg[rs1]→A,*Reg[rs2]→B*	无		打入A和B
M2	A +imm32→F/FR	rs2_imm_s=1,ALU_OP=0000		打入F、FR
M3	PC→Reg[rd] F→PC	w_data_s=3，Reg_Write PC_s=10，PC_Write		打入rd和PC

IS_R+IS_IMM+lui+lw+sw+beq+jal+jalr 指令执行过程

❖目标指令集的25条指令执行的流程图



8.1.5 转移类指令的数据通路设计 发送的控制信号

指令	机器周期	执行操作	发送的控制信号
所有指令	M_0	IMem[PC]→IR，PC→PC0，PC+4→PC	IR_Write，PC_Write，PC0_Write，PC_s=00
R型指令	IS_R·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_R·M_2	A（op）B→F	ALU_OP，rs2_imm_s=0
	IS_R·M_3	F→Reg[rd]	Reg_Write，w_data_s=00
I型运算指令	IS_IMM·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_IMM·M_2	A（op）imm32→F	ALU_OP，rs2_imm_s=1
	IS_IMM·M_3	F→Reg[rd]	Reg_Write，w_data_s=00
lui指令	IS_LUI·M_1	imm32→Reg[rd]	Reg_Write，w_data_s=01
lw指令	IS_LW·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_LW·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_LW·M_3	DMem[F]→MDR	无
	IS_LW·M_4	MDR→Reg[rd]	Reg_Write，w_data_s=10
sw指令	IS_SW·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_SW·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_SW·M_3	B→DMem[F]	Mem_Write
beq指令	IS_BEQ·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_BEQ·M_2	A-B→F/FR	ALU_OP=1000，rs2_imm_s=0
	IS_BEQ·M_3·ZF	PC0+imm32→PC	PC_Write，PC_s=01
jalr指令	IS_JALR·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_JALR·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_JALR·M_3	PC→Reg[rd]，F→PC	Reg_Write，w_data_s=11，PC_Write，PC_s=10
jal指令	IS_JAL·M_1	PC→Reg[rd]，PC0+imm32→PC	Reg_Write，w_data_s=11，PC_Write，PC_s=01

部件的控制信号一览表

❖部件控制信号：均为高电平有效

部件	控制信号	作用	有效边沿
PC	PC_write	写入PC	指定周期下跳沿
PC0	PC0_write	写入PC0	指定周期下跳沿
IR	IR_write	写入IR	指定周期下跳沿
存储器	Mem_Write	存储器写操作	指定周期上跳沿
寄存器堆	Reg_Write	写入寄存器rd	指定周期下跳沿
MDR	无	装入存储器读出数据	每个时钟周期下跳沿
A	无	装入rs1寄存器读出数据	
B	无	装入rs2寄存器读出数据	
F	无	装入ALU运算结果	

8.1.6 RISC-V模型机控制单元CU的设计

- ✓ 功能部件设计
- ✓ 部件连接，建立数据通路
- ✓ 分析指令执行过程、发送的控制信号

❖ 如何产生部件控制信号?

❖ 由操作控制信号形成部件（又称为控制单元CU）产生

❖ CU实现方法：有两种

- 硬布线控制器：组合逻辑电路
  - 输入：IR的opcode、funct3、funct7字段、标志FR、周期信号Mi、节拍Ti
  - 输出：各个控制信号
- 微程序控制器：存储逻辑电路
  - RISC-V：典型的RISC机，指令多为简单指令，故CU不使用微程序实现
- 有限状态机：适用于FPGA器件，本质上组合逻辑+时序逻辑

8.1.6 RISC-V模型机控制单元CU的设计



1、硬布线方式实现控制单元CU

- （1）综合控制信号的逻辑函数
- （2）优化与化简逻辑函数
- （3）时序系统设计

2、有限状态机方式实现CU

- （1）有限状态机原理
- （2）指令执行的状态转移图
- （3）状态编码及输出的控制信号
- （4）控制单元CU的有限状态机

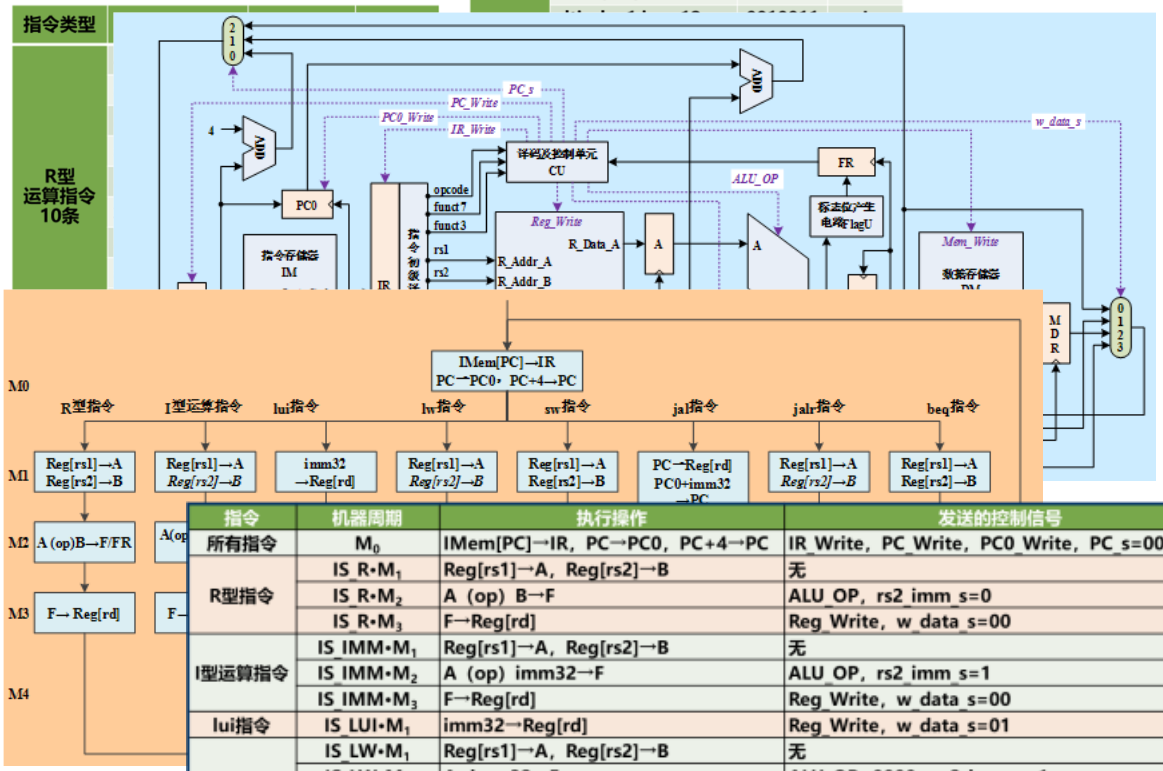
1、硬布线方式实现控制单元CU

- ❖已经完成：
  - 确定目标指令集
  - 确定系统结构和数据通路
  - 分析指令执行过程，写出所有指令各个机器周期发送的控制信号
- ❖接下来：？
  - （1）综合控制信号的逻辑函数
  - （2）优化与化简逻辑函数
  - （3）时序系统设计
  - 电路实现

8.1.1 RISC- V模型机的目标指令集

❖选择实现RV32I的核心指令子集：25条

指令类型	指令	opcode(7)	指令格式
	addi rd,rs1,imm12	0010011	I
	slli rd,rs1,shamt	0010011	I



指令	机器周期	执行操作	发送的控制信号
所有指令	M_0	IMem[PC]→IR，PC→PC0，PC+4→PC	IR_Write，PC_Write，PC0_Write，PC_s=00
R型指令	IS_R·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_R·M_2	A（op）B→F	ALU_OP，rs2_imm_s=0
	IS_R·M_3	F→Reg[rd]	Reg_Write，w_data_s=00
I型运算指令	IS_IMM·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_IMM·M_2	A（op）imm32→F	ALU_OP，rs2_imm_s=1
	IS_IMM·M_3	F→Reg[rd]	Reg_Write，w_data_s=00
lui指令	IS_LUI·M_1	imm32→Reg[rd]	Reg_Write，w_data_s=01
lw指令	IS_LW·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_LW·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_LW·M_3	DMem[F]→MDR	无
	IS_LW·M_4	MDR→Reg[rd]	Reg_Write，w_data_s=10
sw指令	IS_SW·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_SW·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_SW·M_3	B→DMem[F]	Mem_Write
beq指令	IS_BEQ·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_BEQ·M_2	A-B→F/FR	ALU_OP=1000，rs2_imm_s=0
	IS_BEQ·M_3·ZF	PC0+imm32→PC	PC_Write，PC_s=01
jalr指令	IS_JALR·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_JALR·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_JALR·M_3	PC→Reg[rd]，F→PC	Reg_Write，w_data_s=11，PC_Write，PC_s=10
jal指令	IS_JAL·M_1	PC→Reg[rd]，PC0+imm32→PC	Reg_Write，w_data_s=11，PC_Write，PC_s=01

（1）综合控制信号的逻辑函数

与项：指令信号、机器周期信号、状态条件

❖ 控制信号分为三类：

- ①写使能信号：PC_Write、PC0_Write、IR_Write、Reg_Write、Mem_Write
  - 直接将发送这些信号的条件进行“或”运算

指令	机器周期	执行操作	发送的控制信号
所有指令	M_0	IMem[PC]→IR，PC→PC0，PC+4→PC	IR_Write，PC_Write，PC0_Write，PC_s=00
R型指令	IS_R·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_R·M_2	A（op）B→F	ALU_OP，rs2_imm_s=0
	IS_R·M_3	F→Reg[rd]	Reg_Write，w_data_s=00
I型运算指令	IS_IMM·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_IMM·M_2	A（op）imm32→F	ALU_OP，rs2_imm_s=1
	IS_IMM·M_3	F→Reg[rd]	Reg_Write，w_data_s=00
lui指令	IS_LUI·M_1	imm32→Reg[rd]	Reg_Write，w_data_s=01
lw指令	IS_LW·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_LW·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_LW·M_3	DMem[F]→MDR	无
	IS_LW·M_4	MDR→Reg[rd]	Reg_Write，w_data_s=10
sw指令	IS_SW·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_SW·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_SW·M_3	B→DMem[F]	Mem_Write
beq指令	IS_BEQ·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_BEQ·M_2	A-B→F/FR	ALU_OP=1000，rs2_imm_s=0
	IS_BEQ·M_3·ZF	PC0+imm32→PC	PC_Write，PC_s=01
jalr指令	IS_JALR·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_JALR·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_JALR·M_3	PC→Reg[rd]，F→PC	Reg_Write，w_data_s=11，PC_Write，PC_s=10
jal指令	IS_JAL·M_1	PC→Reg[rd]，PC0+imm32→PC	Reg_Write，w_data_s=11，PC_Write，PC_s=01

指令	机器周期	执行操作	发送的控制信号
所有指令	M_0	IMem[PC]→IR，PC→PC0，PC+4→PC	IR_Write，PC_Write，PC0_Write，PC_s=00
R型指令	IS_R·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_R·M_2	A（op）B→F	ALU_OP，rs2_imm_s=0
	IS_R·M_3	F→Reg[rd]	Reg_Write w_data_s=00
I型运算指令	IS_IMM·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_IMM·M_2	A（op）imm32→F	ALU_OP，rs2_imm_s=1
	IS_IMM·M_3	F→Reg[rd]	Reg_Write w_data_s=00
lui指令	IS_LUI·M_1	imm32→Reg[rd]	Reg_Write w_data_s=01
lw指令	IS_LW·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_LW·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_LW·M_3	DMem[F]→MDR	无
	IS_LW·M_4	MDR→Reg[rd]	Reg_Write w_data_s=10
sw指令	IS_SW·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_SW·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_SW·M_3	B→DMem[F]	Mem_Write
beq指令	IS_BEQ·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_BEQ·M_2	A-B→F/FR	ALU_OP=1000，rs2_imm_s=0
	IS_BEQ·M_3·ZF	PC0+imm32→PC	PC_Write，PC_s=01
jalr指令	IS_JALR·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_JALR·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_JALR·M_3	PC→Reg[rd]，F→PC	Reg_Write w_data_s=11，PC_Write，PC_s=10
jal指令	IS_JAL·M_1	PC→Reg[rd]，PC0+imm32→PC	Reg_Write w_data_s=11，PC_Write，PC_s=01

$$PC_Write = M_0 + IS_BEQ \cdot M_3 \cdot ZF + IS_JALR \cdot M_3 + IS_JAL \cdot M_1$$

$$Reg_Write = IS_R \cdot M_3 + IS_IMM \cdot M_3 + IS_LUI \cdot M_1 + IS_LW \cdot M_4 + IS_JALR \cdot M_3 + IS_JAL \cdot M_1$$

（1）综合控制信号的逻辑函数

- 控制信号分为三类：
  - ②多路选择器的选择信号：rs2_imm_s、w_data_s[1:0]、PC_s[1:0]
    - 选择发送“1”信号的条件进行“或”运算即可

指令	机器周期	执行操作	发送的控制信号
所有指令	M_0	IMem[PC]→IR，PC→PC0，PC+4→PC	IR_Write，PC_Write，PC0_Write，PC_s=00
R型指令	IS_R·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_R·M_2	A（op）B→F	ALU_OP，rs2_imm_s=0
	IS_R·M_3	F→Reg[rd]	Reg_Write，w_data_s=00
I型运算指令	IS_IMM·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_IMM·M_2	A（op）imm32→F	ALU_OP，rs2_imm_s=1
	IS_IMM·M_3	F→Reg[rd]	Reg_Write，w_data_s=00
lui指令	IS_LUI·M_1	imm32→Reg[rd]	Reg_Write，w_data_s=01
lw指令	IS_LW·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_LW·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_LW·M_3	DMem[F]→MDR	无
	IS_LW·M_4	MDR→Reg[rd]	Reg_Write，w_data_s=10
sw指令	IS_SW·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_SW·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_SW·M_3	B→DMem[F]	Mem_Write
beq指令	IS_BEQ·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_BEQ·M_2	A-B→F/FR	ALU_OP=1000，rs2_imm_s=0
	IS_BEQ·M_3·ZF	PC0+imm32→PC	PC_Write，PC_s=01
jalr指令	IS_JALR·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_JALR·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_JALR·M_3	PC→Reg[rd]，F→PC	Reg_Write，w_data_s=11，PC_Write，PC_s=10
jal指令	IS_JAL·M_1	PC→Reg[rd]，PC0+imm32→PC	Reg_Write，w_data_s=11，PC_Write，PC_s=01

指令	机器周期	执行操作	发送的控制信号
所有指令	M_0	IMem[PC]→IR，PC→PC0，PC+4→PC	IR_Write，PC_Write，PC0_Write，PC_s=00
R型指令	IS_R·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_R·M_2	A（op）B→F	ALU_OP，rs2_imm_s=0
	IS_R·M_3	F→Reg[rd]	Reg_Write，w_data_s=00
I型运算指令	IS_IMM·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_IMM·M_2	A（op）imm32→F	ALU_OP，rs2_imm_s=1
	IS_IMM·M_3	F→Reg[rd]	Reg_Write，w_data_s=00
lui指令	IS_LUI·M_1	imm32→Reg[rd]	Reg_Write，w_data_s=01
lw指令	IS_LW·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_LW·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_LW·M_3	DMem[F]→MDR	无
	IS_LW·M_4	MDR→Reg[rd]	Reg_Write，w_data_s=10
sw指令	IS_SW·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_SW·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_SW·M_3	B→DMem[F]	Mem_Write
beq指令	IS_BEQ·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_BEQ·M_2	A-B→F/FR	ALU_OP=1000，rs2_imm_s=0
	IS_BEQ·M_3·ZF	PC0+imm32→PC	PC_Write，PC_s=01
jalr指令	IS_JALR·M_1	Reg[rs1]→A，Reg[rs2]→B	无
	IS_JALR·M_2	A+imm32→F	ALU_OP=0000，rs2_imm_s=1
	IS_JALR·M_3	PC→Reg[rd]，F→PC	Reg_Write，w_data_s=11，PC_Write，PC_s=10
jal指令	IS_JAL·M_1	PC→Reg[rd]，PC0+imm32→PC	Reg_Write，w_data_s=11，PC_Write，PC_s=01

$$rs2_imm_s = IS_IMM \cdot M_2 + IS_LW \cdot M_2 + IS_SW \cdot M_2 + IS_JALR \cdot M_2$$
$$w_data_s[1] = IS_LW \cdot M_4 + IS_JALR \cdot M_3 + IS_JAL \cdot M_1$$
$$w_data_s[0] = IS_LUI \cdot M_1 + IS_JALR \cdot M_3 + IS_JAL \cdot M_1$$

（1）综合控制信号的逻辑函数

- 控制信号分为三类：
  - ③ALU运算码：ALU_OP[3:0]
    - 综合后可得各位的逻辑函数

$$ALU_OP[0]=(IS_R+IS_IMM)\cdot funct3[0]$$
$$ALU_OP[1]=(IS_R+IS_IMM)\cdot funct3[1]$$
$$ALU_OP[2]=(IS_R+IS_IMM)\cdot funct3[2]$$

输入：opcode	输出：指令信息	输出：ALU_OP
0110011	IS_R	{funct7[5],funct3}
0010011	IS_IMM	funct3≠101：{1’b0,funct3} funct3=101：{funct7[5],funct3}
0110111	IS_LUI	xxxx
0000011	IS_LW	0000
0100011	IS_SW	0000
1101111	IS_JAL	0000
1100111	IS_JALR	0000
1100011	IS_BEQ	1000

$$ALU_OP[3]=funct7[5]\cdot(IS_R+IS_IMM\cdot funct3[2]\cdot\overline{funct3[1]}\cdot funct3[0])+IS_BEQ$$

（1）综合控制信号的逻辑函数

- IR_write = M_0
- PC0_write = M_0
- PC_write = M_0 + IS_BEQ•M_3•ZF + IS_JAL•M_1 + IS_JALR •M_3
- Reg_write = IS_R•M_3 +IS_IMM•M_3 +IS_LUI•M_1 +IS_LW•M_4 +IS_JAL•M_1 + IS_JALR•M3
- Mem_write = IS_SW•M3
- rs2_imm_s = IS_IMM•M_2 + IS_LW •M_2 + IS_SW •M_2 + IS_JALR •M_2
- w_data_s[1] = IS_LW•M_4 + IS_JAL •M_1 + IS_JALR •M_3
- w_data_s[0] = IS_LUI•M_1 + IS_JAL •M_1 + IS_JALR •M_3
- PC_s[1] = IS_JALR •M_3
- PC_s[0] = IS_BEQ •M_3•ZF + IS_JAL •M_1

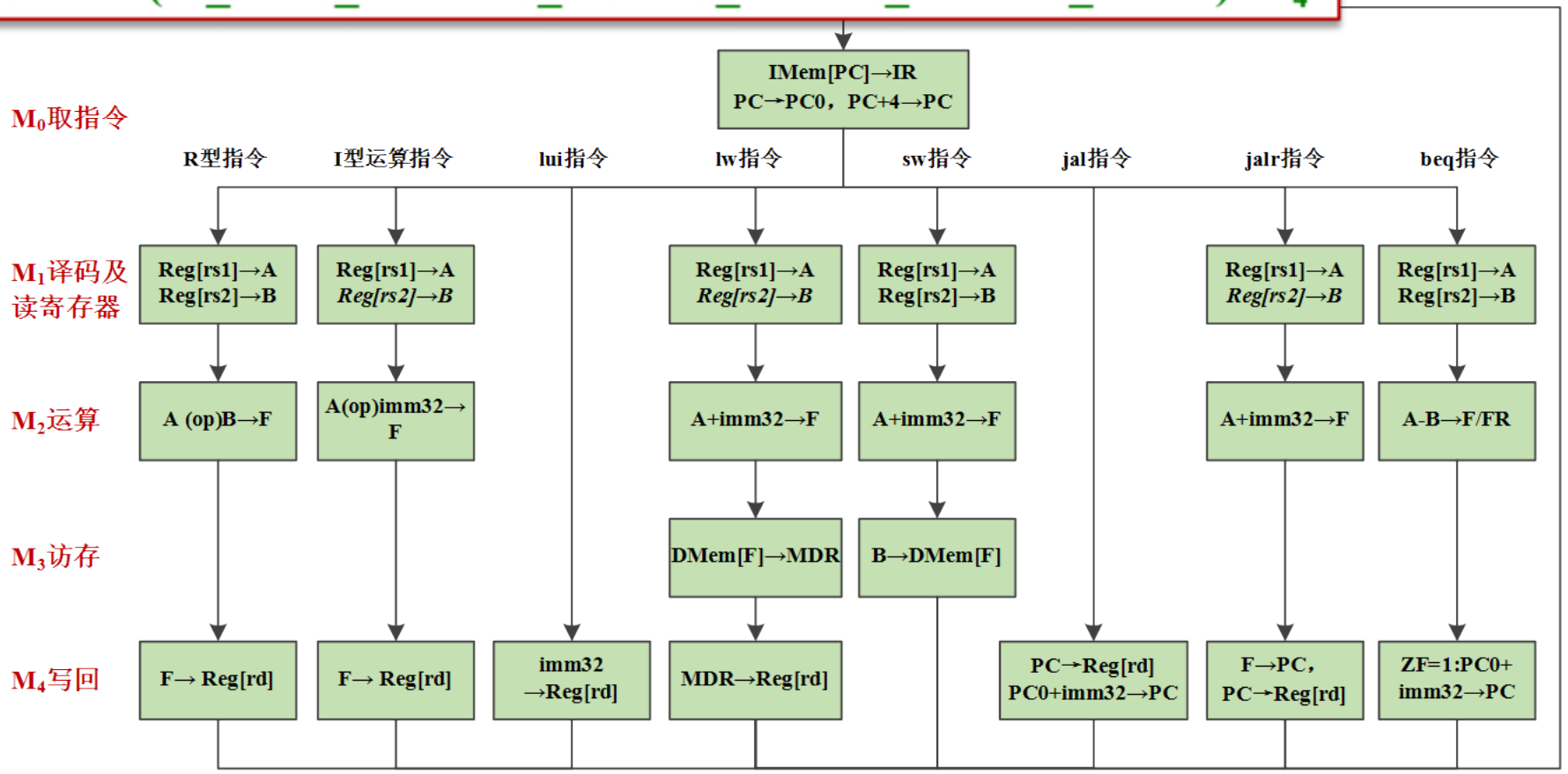
（2）优化与化简逻辑函数

❖ 思路1：如果某个控制信号在同一个机器周期发送，势必会使得逻辑函数简化

- 调整：写回或转移操作都在M_4周期执行

$$Reg_Write = IS_R \cdot M_3 + IS_IMM \cdot M_3 + IS_LUI \cdot M_1 + IS_LW \cdot M_4 + IS_JALR \cdot M_3 + IS_JAL \cdot M_1$$

$$Reg_Write = (IS_R+IS_IMM+IS_LUI+IS_LW+IS_JAL+IS_JALR) \cdot M_4$$



（2）优化与化简逻辑函数

- 思路2：将目标指令系统看做一个整体（逻辑1），多数指令发送的信号，可以用少数指令的“非”来代替
  - 例如：

$$Reg_Write = (IS_R+IS_IMM+IS_LUI+IS_LW+IS_JAL+IS_JALR)\cdot M_4$$

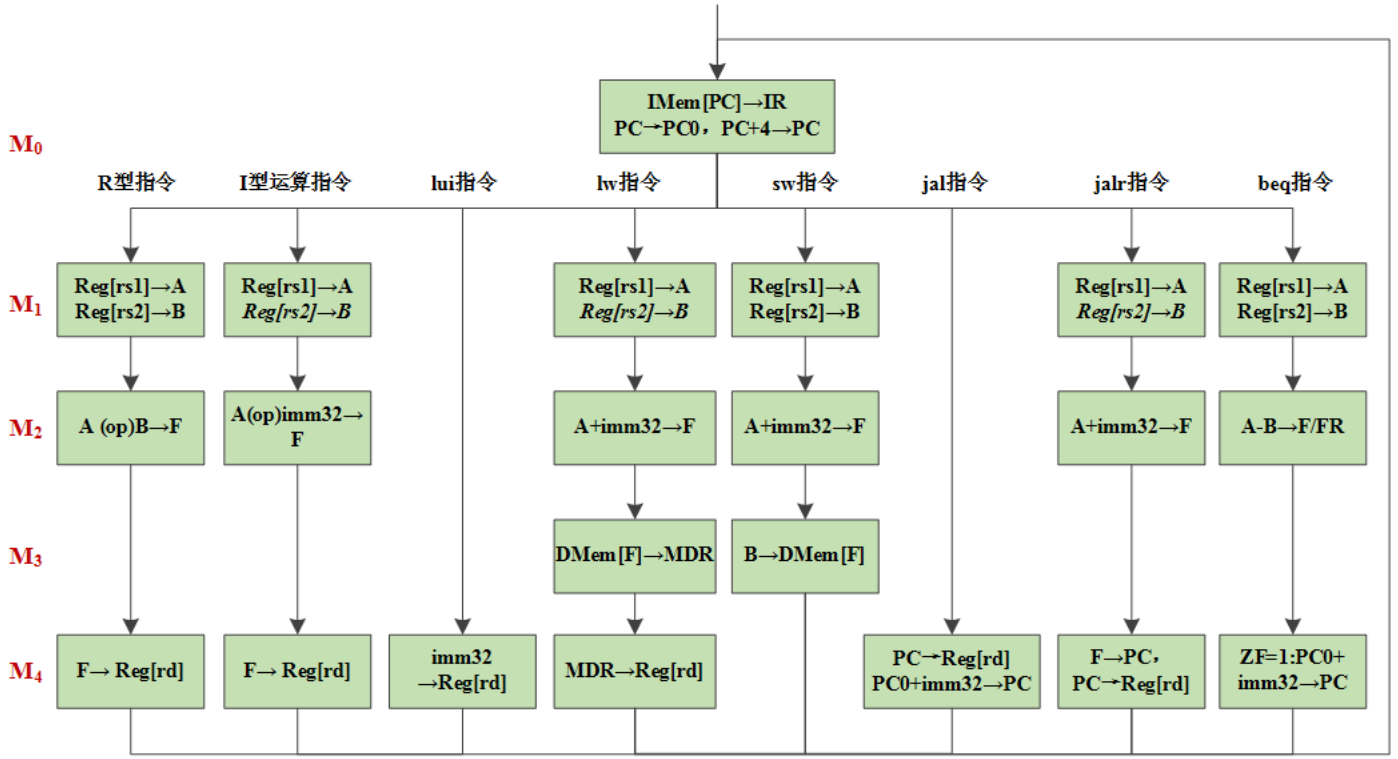
- 分析：
  - M_4周期：只有beq指令和sw指令没有发送Reg_Write信号
  - sw指令：根本没有M_4周期
  - 简化Reg_Write函数：

$$Reg_Write = \overline{IS_BEQ}\cdot M_4$$

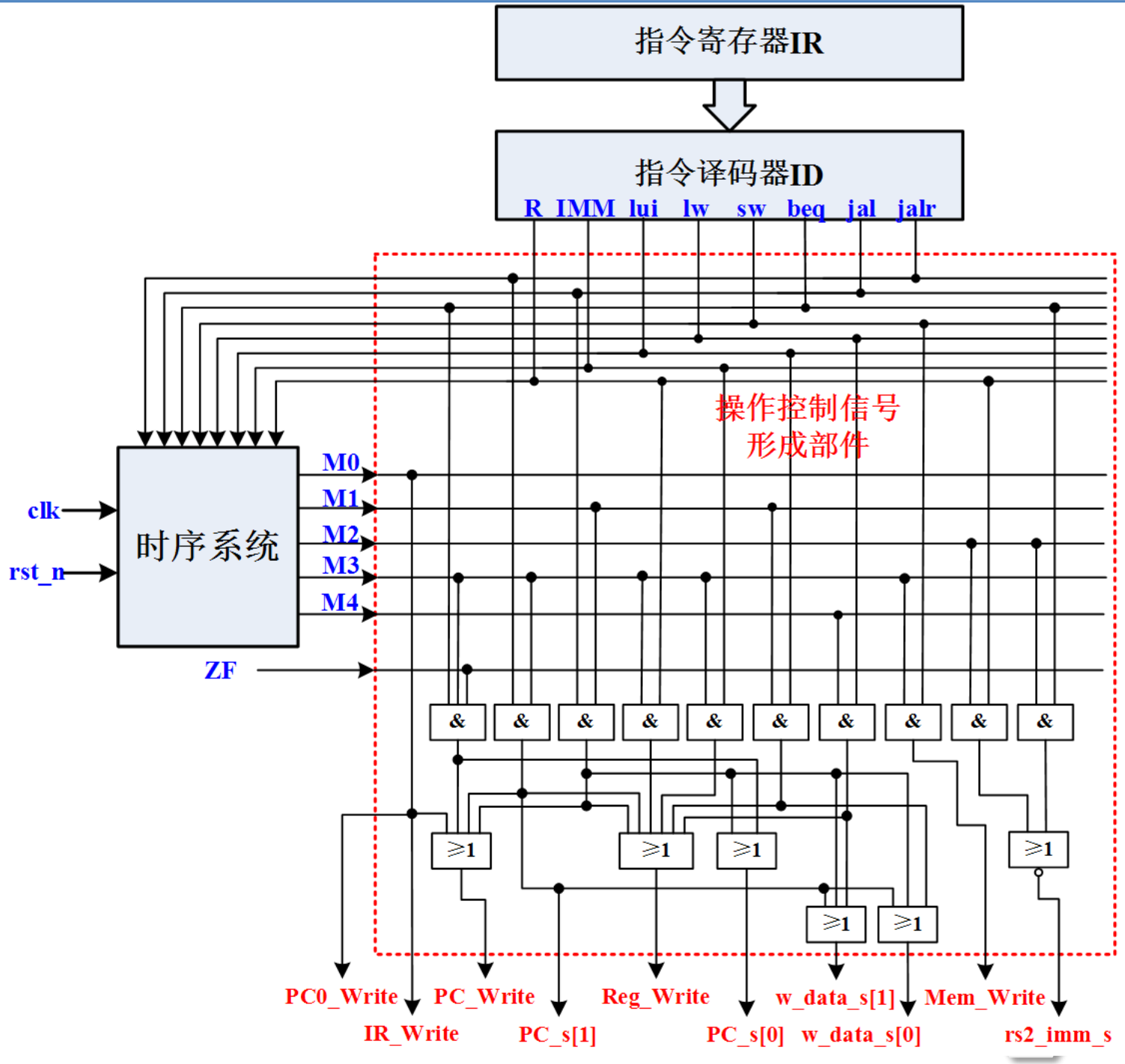
（3）时序系统设计

- 时序系统设计目标：按照各指令的机器周期顺序产生信号M_i
  - R型指令：$M_0 \rightarrow M_1 \rightarrow M_2 \rightarrow M_4 \rightarrow M_0$
  - lui指令：$M_0 \rightarrow M_4 \rightarrow M_0$
  - ……
- 设计方法和案例见7.1.3节和7.3.2节
  - 使用计数器+译码器实现
  - 计数器的状态转换顺序，取决于指令信号
    - R型指令：000→001→010→100→000
    - lui指令：000→100→000

- 时序系统电路
  - 输入：指令信号
  - 输出：机器周期信号序列



电路实现


指令寄存器IR
指令译码器ID
R IMM lui lw sw beq jal jalr
操作控制信号
形成部件
clk
rst_n
时序系统
M0
M1
M2
M3
M4
ZF
&
≥1
PC0_Write
IR_Write
PC_Write
PC_s[1]
Reg_Write
PC_s[0]
w_data_s[1]
w_data_s[0]
Mem_Write
rs2_imm_s

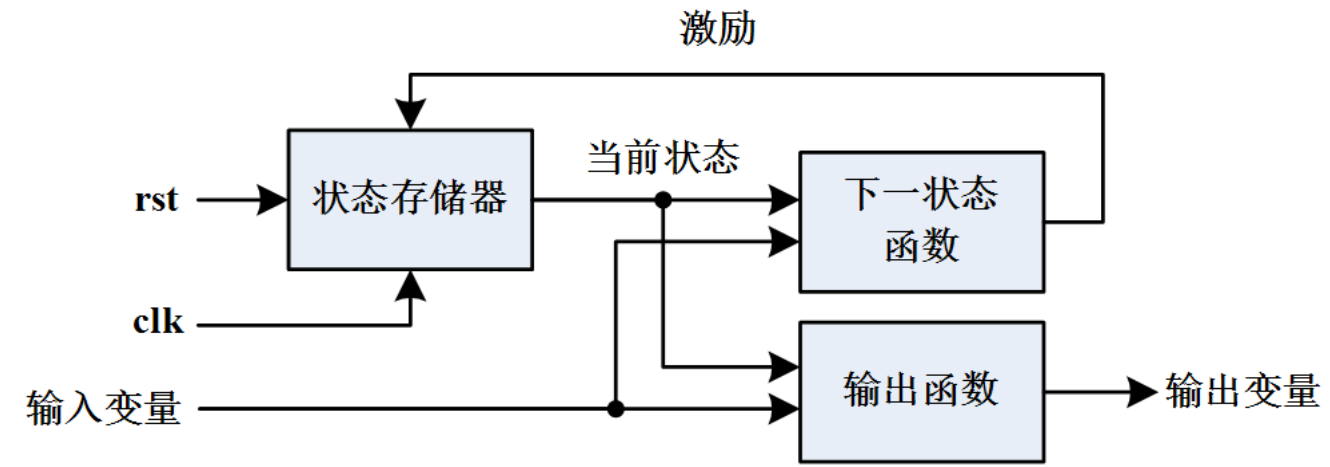
2、有限状态机方式实现CU

❖（1）有限状态机原理

❖ 有限状态机FSM：Finite State Machine，通过状态迁移来完成特定的顺序控制逻辑

❖ 构成：1个状态存储器+2个函数

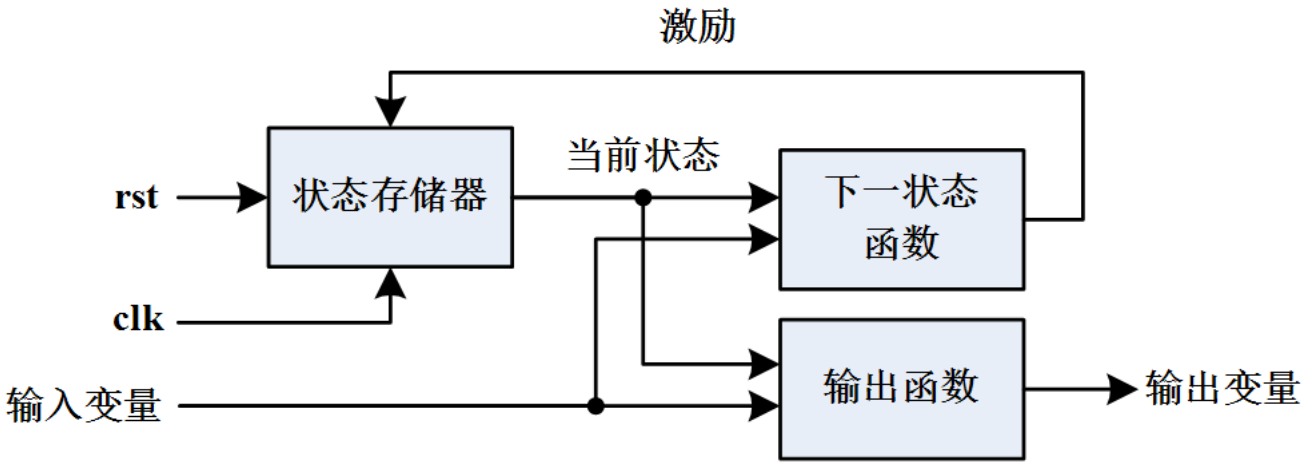
- 状态存储器：由触发器构成，保存FSM的当前状态
- 下一状态函数：根据输入和当前状态来产生状态触发器的激励信号，实现向次态的转移
- 输出函数：由当前状态、当前输入产生状态机的输出信号
- 复位信号rst：初始化有限状态机为初始状态
- 时钟信号clk：触发状态转换



（1）有限状态机原理

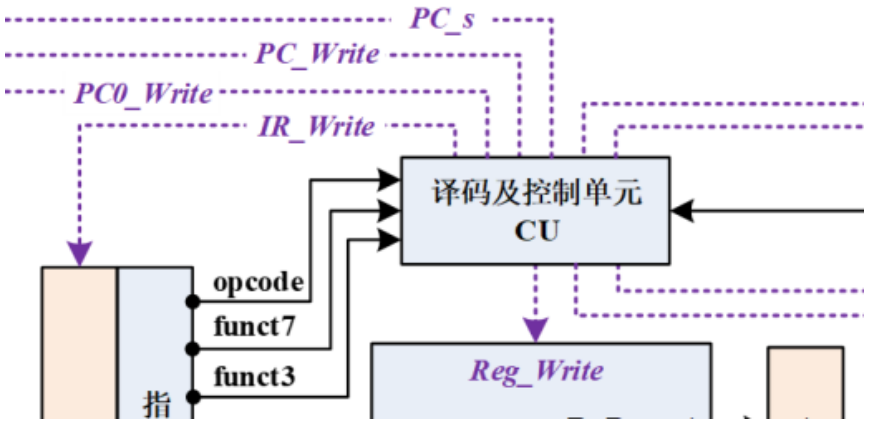
❖ 有限状态机设计步骤：

- ① 确定电路的输入、输出与状态，画出状态转移图
- ② 状态化简：得到最简的状态转移图
- ③ 状态编码：可以使用自然二进制、格雷码或独热码等编码方式
- ④ 求出下一状态函数和输出函数，用电路实现；或者采用HDL描述，用FPGA等器件实现。



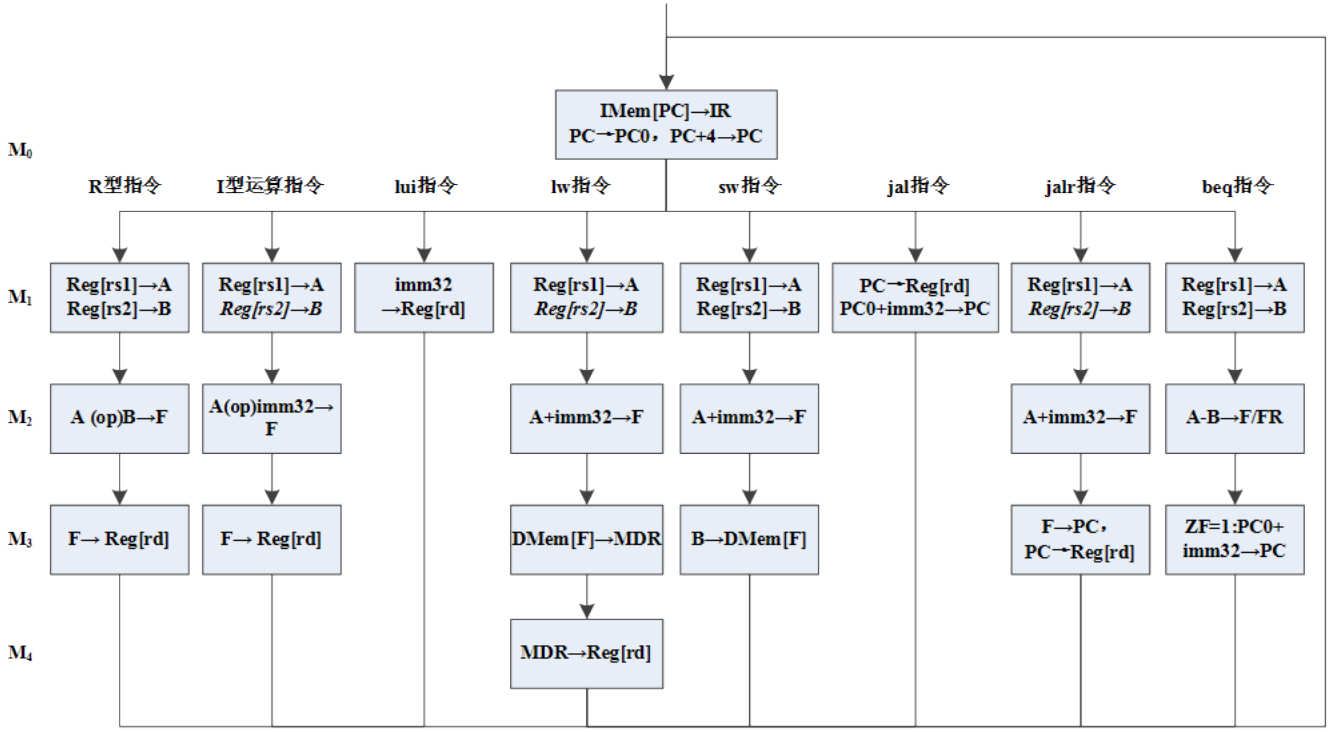
❖ 如果用有限状态机实现控制单元CU：

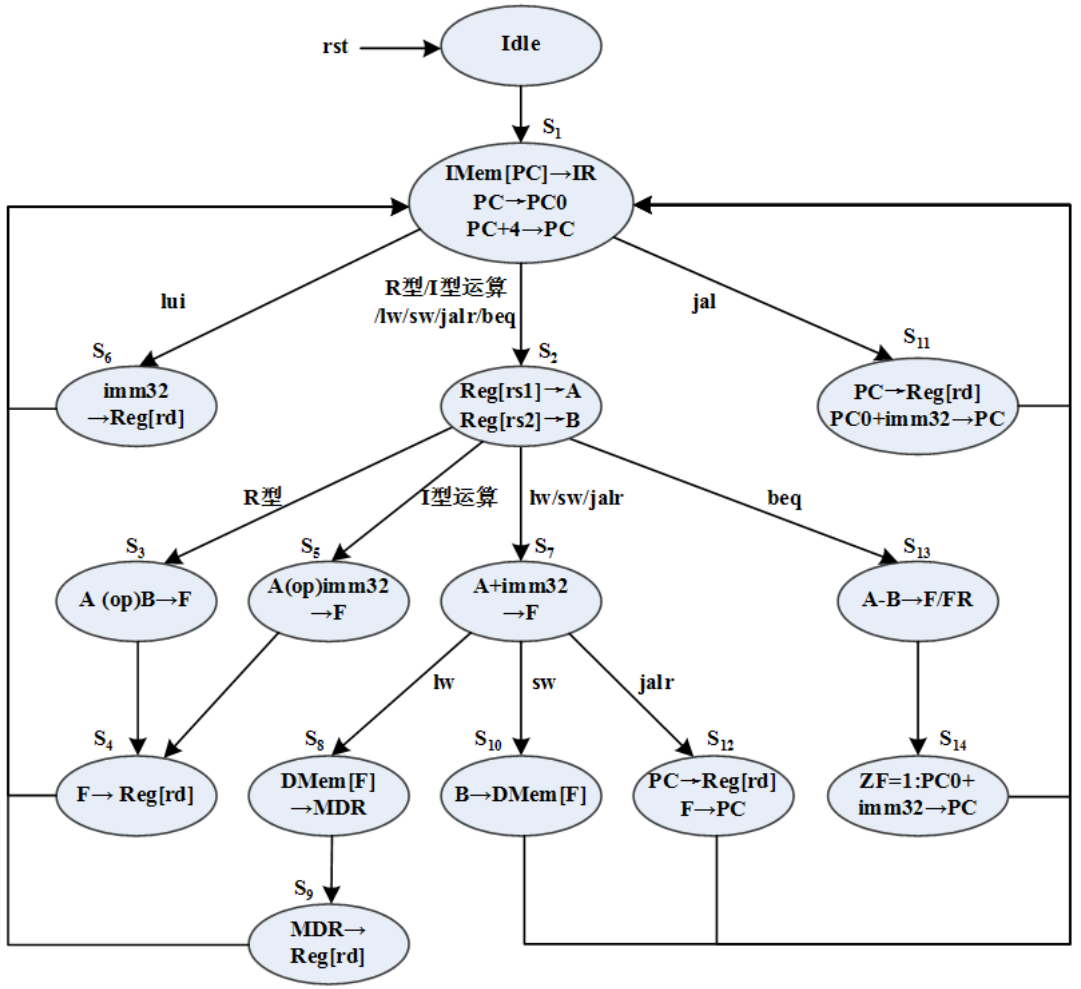
- 输入变量：指令译码输出的指令信号（IS_R、IS_IMM等）、FR的状态
- 输出变量：指令要发送的控制信号；什么时间、发送哪些信号，则由当前状态决定。
- 状态：25条目标指令集的RISC-V模型



（2）指令执行的状态转移图

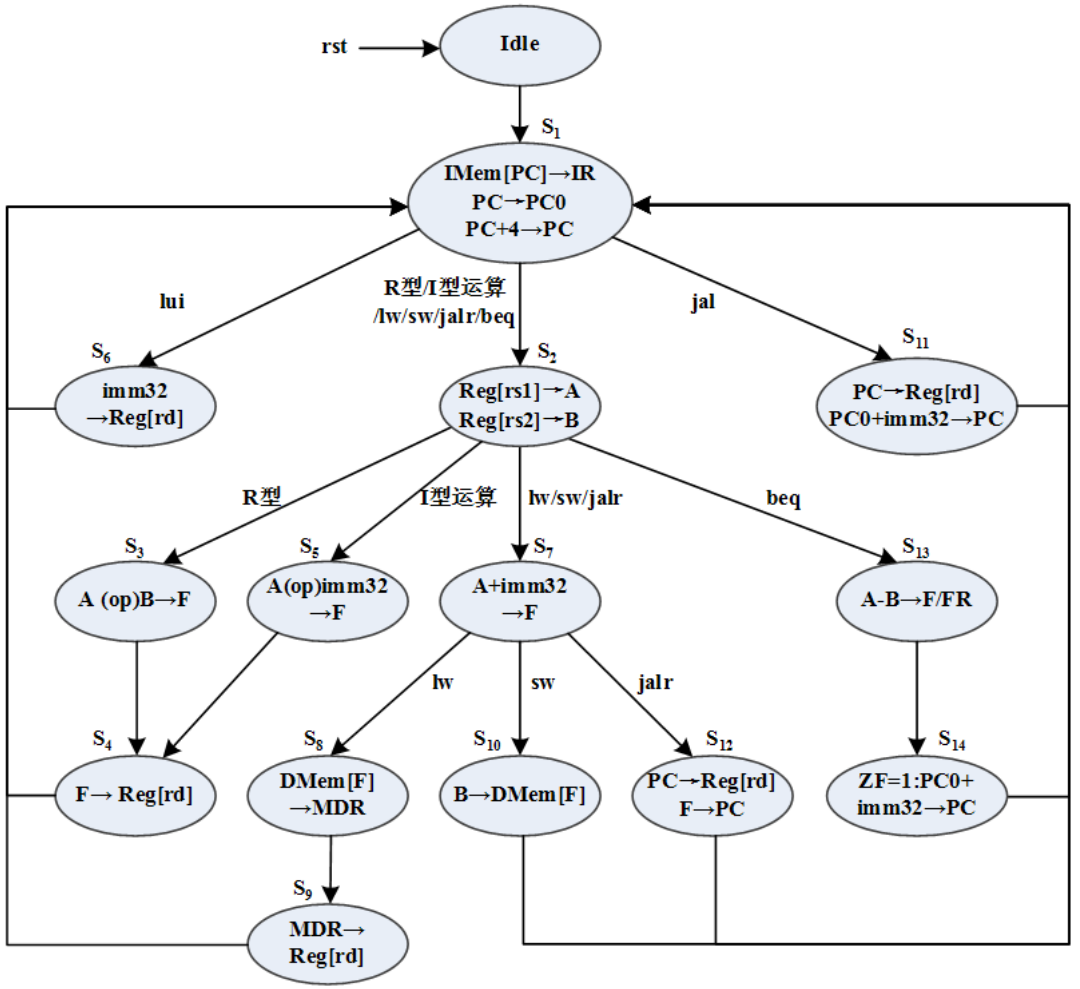
- **指令执行流程图：就是一个状态转移图，每个方框就是一个状态。**
  - 有重复状态：例如“Reg[rs1]→A，Reg[rs2]→B”
  - 合并重复的状态：得到最简状态转移图
- **RISC-V模型机：**
  - 21个状态：合并成15个状态Idle、S_1~S_{14}





（2）指令执行的状态转移图

- 线的箭头：状态转移的方向
- 转移条件：指令信号，标注在线上
  - 线上空白：表明无条件必转往下一个状态
- Idle状态：空闲状态，初始状态
- S_1状态：取指令状态
  - S_1次态：根据指令译码结果进行分支
    - 三个分支：lui、jal、其他指令
- 每条指令的最后一个状态，都回到S_1
  - R型指令状态转移：$S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_4 \rightarrow S_1$
  - jalr指令状态转移：$S_1 \rightarrow S_2 \rightarrow S_7 \rightarrow S_{12} \rightarrow S_1$
  - lw指令最长：$S_1 \rightarrow S_2 \rightarrow S_7 \rightarrow S_8 \rightarrow S_9 \rightarrow S_1$



（3）状态编码及输出的控制信号

- 15个状态：采用4位二进制编码
  - Idle编码为0000，S_1编码为0001，S_2编码为0010……
- 分析每个状态要发送的控制信号

- “—”：表示该信号在此状态下是无关的，即无论取何值都不影响该状态要执行的操作
- 写控制信号：需要写入，则=1；否则=0
- Idle：写控制信号全部无效，其他信号全部无关
- ALU_OP=****：代表由指令操作码和功能码产生
- ALU_OP=—：无关，ALU运算结果不被后续状态使用

状态 / 控制信号	Idle	S_1	S_2	S_3	S_4	S_5	S_6	S_7	S_8	S_9	S_{10}	S_{11}	S_{12}	S_{13}	S_{14}
PC_Write	0	1	0	0	0	0	0	0	0	0	0	1	1	0	ZF
PC0_Write	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
IR_Write	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
Reg_Write	0	0	0	0	1	0	1	0	0	1	0	1	1	0	0
Mem_Write	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
ALU_OP	—	—	—	****	—	****	—	0000	—	—	—	—	—	1000	—
rs2_imm_s	—	—	—	0	—	1	—	1	—	—	—	—	—	0	—
w_data_s[1:0]	—	—	—	—	00	—	01	—	—	10	—	11	11	—	—
PC_s[1:0]	—	00	—	—	—	—	—	—	—	—	—	01	10	—	01

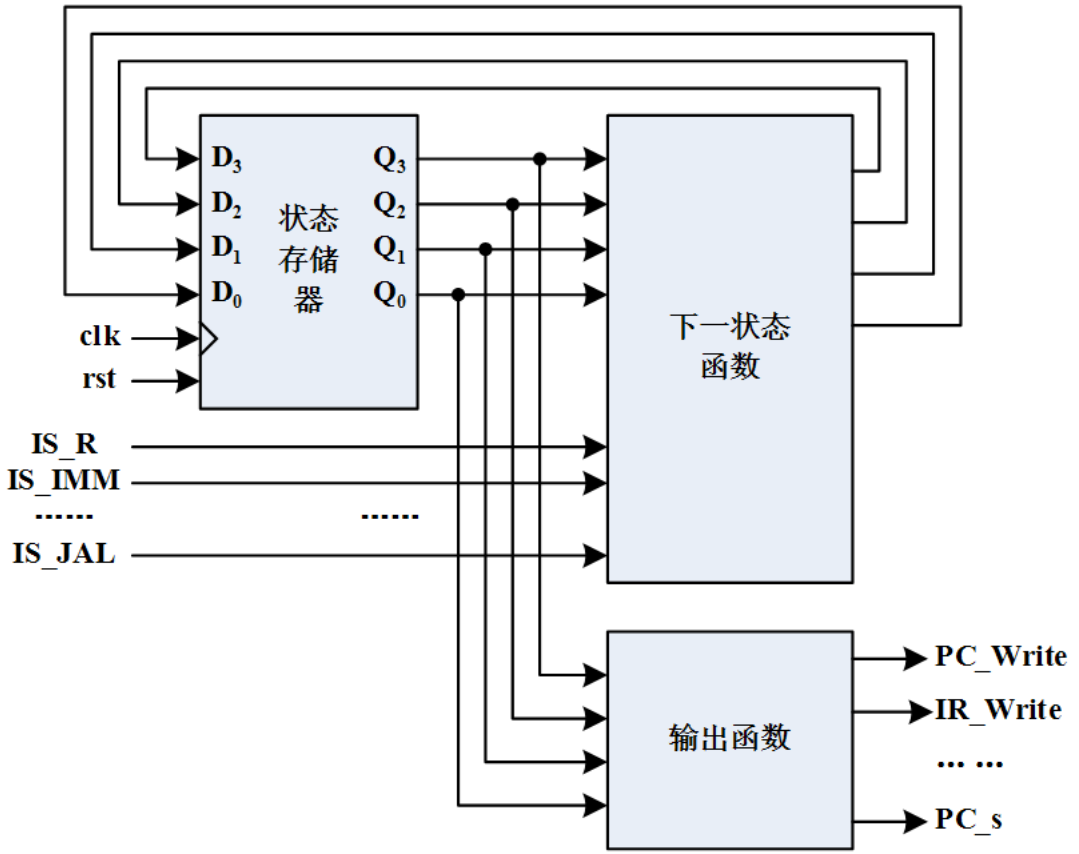
（3）状态编码及输出的控制信号

- S_1状态：IMem[PC]→IR,PC→PC0,PC+4→PC
  - 置PC_Write、PC0_Write、IR_Write为1
  - 置Reg_Write和Mem_Write为0
  - ALU无论执行什么运算、对谁运算，结果都不会改变寄存器或存储器内容：ALU_OP、rs2_imm_s和w_data_s都无关
- beq状态S_{14}：如果ZF=1，则PC0+imm32→PC
  - ZF=1：相等，则转移，设置PC_Write=1，PC_s=01
  - ZF=0：不等，则不转移，设置PC_Write=0，PC_s=xx
  - 综合：用ZF产生PC_Write，即PC_Write=ZF，PC_s取01

状态 / 控制信号	Idle	S_1	S_2	S_3	S_4	S_5	S_6	S_7	S_8	S_9	S_{10}	S_{11}	S_{12}	S_{13}	S_{14}
PC_Write	0	1	0	0	0	0	0	0	0	0	0	1	1	0	ZF
PC0_Write	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
IR_Write	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
Reg_Write	0	0	0	0	1	0	1	0	0	1	0	1	1	0	0
Mem_Write	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
ALU_OP	—	—	—	****	—	****	—	0000	—	—	—	—	—	1000	—
rs2_imm_s	—	—	—	0	—	1	—	1	—	—	—	—	—	0	—
w_data_s[1:0]	—	—	—	—	00	—	01	—	—	10	—	11	11	—	—
PC_s[1:0]	—	00	—	—	—	—	—	—	—	—	—	01	10	—	01

（4）控制单元CU的有限状态机

- 控制单元CU的有限状态机：
  - 状态存储器：由4个D触发器构成（状态编码4位）
  - 下一状态函数：对照状态转移图→列出状态转换真值表
    - 输入：状态存储器现态、指令信号
    - 输出：触发器次态的激励输入（D触发器的次态等于激励）
  - 输出函数：
    - 输入：状态编码
    - 输出：各个控制信号



（4）控制单元CU的有限状态机

❖ 使用FPGA器件实现有限状态机：三段式描述

```
//第一段：状态转移,clk边沿触发,同步时序逻辑电路
always @(negedge rst_n or posedge clk)
begin
   if (!rst_n)ST <= Idle;  //初始状态
   else  ST <= Next_ST;  //clk上沿，状态转移
end
```

```
//第二段：次态函数,阻塞式赋值,是组合逻辑函数
always @(*)
begin
   Next_ST = Idle;
   case (ST)       //根据状态转移图进行次态赋值
      Idle:  Next_ST = S1;
      S1: begin
         if (IS_LUI) Next_ST = S6;
         else  if(IS_JAL) Next_ST = S11;
         else  Next_ST = S2;
      end
      ......
      default:  Next_ST = S1;
   endcase
end
```

❖ 进入新状态的同时，相应的控制信号也同时有效

```
//第三段：输出函数，clk上跳沿，根据下一状态进行控制信号的非阻塞式赋值
always @(negedge rst_n or posedge clk)
begin
   if (!rst_n)  begin    //全部信号初始化为0
      PC_write <= 1'b0;   IR_write <= 1'b0;
      ......
   end
   else  begin
     case(Next_ST)
        S1:begin    //信号的非阻塞式赋值
           PC_write<=1'b1;
           IR_write<=1’ b1; ......
        end
        ......//每个状态下输出的控制信号
     endcase
  end;
end
```

本章小结

- RISC-V模型机：
  - 目标指令集：RV32I的典型指令子集
  - 系统结构：多周期、直接连接
  - RISC-V模型机的组成部件
  - 各类指令的数据通路、指令的执行过程
    - 取指令、指令译码/读寄存器、执行计算、访问存储器、写回寄存器5个阶段
  - 控制单元CU的设计

本章小结

- ARM模型机：
  - 目标指令集
  - 系统结构：多周期、直接连接
  - ARM模型机的组成部件
  - 5类指令的数据通路
  - 简单的ARM硬布线控制单元CU的设计实例
  - 能够模拟实现按条件取指令、9种工作模式切换、块数据传送等多种具有ARM特色的底层技术

The End !